

Incidentes de ciberseguridad

Práctica de implantación de ELK

Autor: Rodríguez Rubén

ÍNDICE:

Introducción:	2
Paso 0º Preparación del entorno	3
Paso 0.1 Instalación.....	4
Paso 0.1.2 Configuración de red.....	5
Paso 0.1.3 Instalación de docker.....	7
Paso 1º Implantación del ELK	12
Paso 1.2 Configuración de Windows para Kibana (con winlogbeat).....	15
Paso 1.2.1 Configuración del servicio winlogbeat.....	17
Paso 2º Visualización y análisis de eventos en Kibana	21
Paso 2.1 Discover.....	23
Paso 2.2 HTTPS en la comunicación de métricas.....	24
Paso 2.3 Gestión segura de credenciales en winlogbeat.....	27
Paso 3º Configuración de alertas en Kibana	29
Paso 3.1 Creación de alertas de seguridad en Kibana (detección de fuerza bruta).....	31
Paso 3.1.1 Comprobación de la efectividad de la regla de detención.....	35
Paso 3.2 Mandar alerta vía Correo.....	38
Paso 3.2.1 Mandar correo de Alerta cuándo se activa la regla.....	44
Paso 3.2.1.1 Configuración de los parámetros para evitar falsos positivos.....	53
Paso 4º. Creación de Grafos	59
Extensión: SIEM en DOCKER	62
E.1 Preparación del fichero de implantación	63
Conclusión:	64

Introducción:

En el presente ejercicio se llevará a cabo la implementación del stack ELK (Elasticsearch, Logstash y Kibana) utilizando Docker sobre una máquina virtual.

El objetivo es desplegar y configurar cada uno de los componentes de manera adecuada, garantizando su integración y correcto funcionamiento.

Finalmente, se realizarán pruebas de validación para comprobar la eficacia de la solución y evaluar su rendimiento en un entorno controlado.

Paso 0º Preparación del entorno.

En este apartado se procederá a la creación y configuración del entorno de trabajo que servirá como base para la implementación del stack **ELK**. El objetivo es garantizar que la máquina virtual disponga de los recursos, dependencias y herramientas necesarias para desplegar los contenedores de **Elasticsearch, Logstash y Kibana** mediante **Docker**.

La preparación incluye:

- Definir los requisitos mínimos de la máquina virtual (CPU, memoria RAM y almacenamiento).
- Instalar y actualizar los paquetes esenciales del sistema operativo.
- Configurar **Docker** y **Docker Compose** para la gestión de contenedores.
- Crear la estructura inicial del proyecto que se utilizará en los pasos posteriores.

Requisitos de las VM.

	ELK Server	Cliente
SO	Ubuntu Server	Windows 10
CPU	2	2
RAM	8GB	2GB
Almacenamiento	50 GB	40 GB

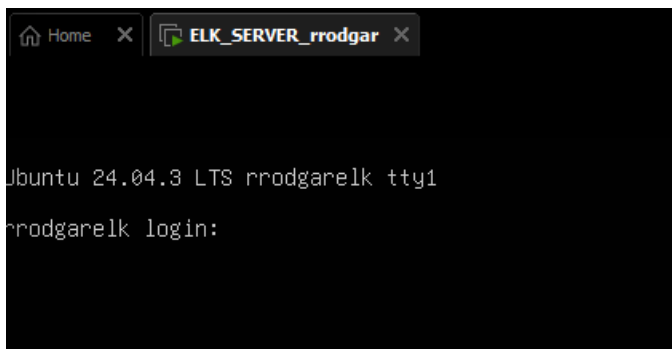
Paso 0.1 Instalación

En este paso se instalarán las herramientas necesarias para desplegar el stack ELK sobre la máquina virtual ya creada. No se detallará el proceso de creación de la VM, al considerarse fuera del alcance de esta práctica.

Las acciones principales son:

- Actualizar el sistema operativo.
- Instalar Docker y Docker Compose.
- Configurar permisos de usuario para ejecutar contenedores.
- Verificar que el entorno está listo para la implementación.

Una vez tengamos la máquina virtual servidor, tendremos que acceder desde SSH, para hacerlo realista, y por supuesto de forma segura.

A terminal window titled 'ELK_SERVER_rrodgar' showing the login process for Ubuntu 24.04.3 LTS. The prompt is 'rrodgarelk login:' and the user has entered 'rrodgarelk'.

Aquí tenemos nuestro servidor en marcha, recién instalado.

Una vez instalamos el sistema, lo primero es actualizar el repositorio, para poder instalar docker en el próximo paso.

```
usuario@rrodgarelk:~$ sudo apt update
[sudo] password for usuario:
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Fetched 126 kB in 1s (145 kB/s)
```

Paso 0.1.2 Configuración de red.

Cómo bien sabemos, un servidor como es el nuestro por ejemplo, debe tener una ip fija, para evitar contratiempos, por lo que vamos a poner una ip fija dentro del rango de vmware y de la red claro.

Nos vamos a:

```
usuario@rrodgarelk:~$ sudo nano /etc/netplan/50-cloud-init.yaml _
```

Y entramos...

```
GNU nano 7.2 /etc/netplan/50-cloud-init.yaml
network:
  version: 2
  ethernet:
    ens33:
      dhcp4: true
```

Tenemos que modificar el ens33, es cierto que lo recomendable es usar dos interfaces, la nat para descargar paquetes, y otra para el cliente y la comunicación, yo voy a usar NAT por el momento.

```
GNU nano 7.2 /etc/netplan/50-cloud-init.yaml
network:
  version: 2
  ethernet:
    ens33:
      dhcp4: no
      addresses: [192.168.65.120/24]
      nameservers:
        addresses: [8.8.8.8,1.1.1.1]
      routes:
        - to: default
          via: 192.168.65.2
```

Ponemos los dns y la puerta de enlace (routes).

Aplicamos los cambios...

```
usuario@rrodgarelk:~$ sudo netplan apply
usuario@rrodgarelk:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:e5:fc:22 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.65.120/24 brd 192.168.65.255 scope global ens33
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fee5:fc22/64 scope link
        valid_lft forever preferred_lft forever
usuario@rrodgarelk:~$
```

Vemos que tenemos la ip, para ver si tenemos conectividad al exterior, haré un ping a google.

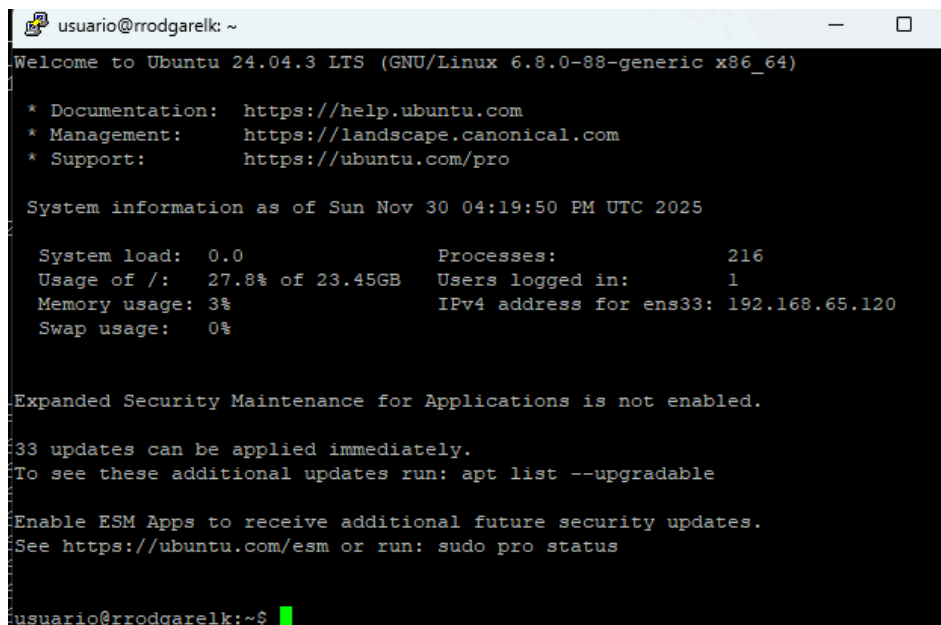
```
usuario@rrodgarelk:~$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=128 time=11.2 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=128 time=11.1 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=128 time=11.1 ms
_
```

Hay conexión.

Paso 0.1.3 Instalación de docker.

En este paso vamos a instalar Docker en nuestro servidor, docker en pocas palabras es un software que nos permitirá correr micro servicios o aplicaciones de forma ligera y lo más importante ... de forma rápida.

Yo usaré ssh, que es lo que recomiendo, por defecto SSH tiene unos ajustes, que podemos tocar, pero que nos basta con la de por defecto.



```
usuario@rrodgarelk: ~  
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-88-generic x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:        https://ubuntu.com/pro  
  
System information as of Sun Nov 30 04:19:50 PM UTC 2025  
  
System load:  0.0                Processes:            216  
Usage of /:   27.8% of 23.45GB   Users logged in:     1  
Memory usage: 3%                IPv4 address for ens33: 192.168.65.120  
Swap usage:   0%  
  
Expanded Security Maintenance for Applications is not enabled.  
  
33 updates can be applied immediately.  
To see these additional updates run: apt list --upgradable  
  
Enable ESM Apps to receive additional future security updates.  
See https://ubuntu.com/esm or run: sudo pro status  
  
usuario@rrodgarelk:~$
```

A partir de aquí empezamos la instalación de docker.

Para ello me apoyaré de la documentación oficial: [Instalación docker en ubuntu](#)

Iré siguiendo los pasos.

Primero añadir una serie de key o llave de repositorios de docker para las dependencias.


```

usuario@rrodgarelk:~$ # Add Docker's official GPG key:
2sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyring
s/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

U# Add the repository to Apt sources:
Fsudo tee /etc/apt/sources.list.d/docker.sources <<EOF
6Types: deb
6URIs: https://download.docker.com/linux/ubuntu
6Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
6Components: stable
6Signed-By: /etc/apt/keyrings/docker.asc
6EOF
6
6sudo apt update

```

Esperamos...

```

Building dependency tree... Done
Reading state information... Done
33 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ca-certificates is already the newest version (20240203).
ca-certificates set to manually installed.
curl is already the newest version (8.5.0-2ubuntu10.6).
curl set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 33 not upgraded.
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: noble
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
Get:1 https://download.docker.com/linux/ubuntu noble InRelease [48.5 kB]
Hit:2 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Get:6 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages
kB]
0% [Working]

```

Terminó...

```

kB]
Fetched 87.8 kB in 0s (176 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
33 packages can be upgraded. Run 'apt list --upgradable' to see them.
usuario@rrodgarelk:~$

```

Instalamos docker y sus componentes cómo compose y plugins.

```
usuario@rrodgarelk: ~  
usuario@rrodgarelk:~$ sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plug  
in
```

```
usuario@rrodgarelk: ~  
usuario@rrodgarelk:~$ sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plug  
in  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
The following additional packages will be installed:  
  docker-ce-rootless-extras libslirp0 pigz slirp4netns  
Suggested packages:  
  cgroupfs-mount | cgroup-lite docker-model-plugin  
The following NEW packages will be installed:  
  containerd.io docker-buildx-plugin docker-ce docker-ce-cli docker-ce-rootless-extras docker-compose-plugin  
  libslirp0 pigz slirp4netns  
0 upgraded, 9 newly installed, 0 to remove and 33 not upgraded.  
Need to get 97.8 MB of archives.  
After this operation, 409 MB of additional disk space will be used.  
Do you want to continue? [Y/n] Y  
Get:1 https://download.docker.com/linux/ubuntu noble/stable amd64 containerd.io amd64 2.2.0-2~ubuntu.24.04~noble [23.  
3 MB]  
Get:2 http://archive.ubuntu.com/ubuntu noble/universe amd64 pigz amd64 2.8-1 [65.6 kB]  
Get:3 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-ce-cli amd64 5:29.1.1-1~ubuntu.24.04~noble [1  
16.3 MB]  
Get:4 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-ce amd64 5:29.1.1-1~ubuntu.24.04~noble [21.0  
MB]  
Get:5 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-buildx-plugin amd64 0.30.1-1~ubuntu.24.04~no  
ble [16.4 MB]  
Get:6 http://archive.ubuntu.com/ubuntu noble/main amd64 libslirp0 amd64 4.7.0-1ubuntu3 [63.8 kB]  
Get:7 http://archive.ubuntu.com/ubuntu noble/universe amd64 slirp4netns amd64 1.2.1-1build2 [34.9 kB]  
Get:8 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-ce-rootless-extras amd64 5:29.1.1-1~ubuntu.2  
4.04~noble [6,381 kB]  
Get:9 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-compose-plugin amd64 2.40.3-1~ubuntu.24.04~n  
oble [14.3 MB]  
Fetched 97.8 MB in 1s (69.7 MB/s)  
Selecting previously unselected package containerd.io.  
[Reading database ... 55%
```

Tras terminar la instalación, debemos de meter nuestro usuario en el grupo de docker, porque si por ejemplo queremos ver los contenedores corriendo....pasará esto.

```
usuario@rrodgarelk:~$ docker ps -a  
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock  
usuario@rrodgarelk:~$
```

Porque no tenemos privilegios.

Vamos a ejecutar este comando....

```
usuario@rrodgarelk:~$ sudo usermod -aG docker usuario  
usuario@rrodgarelk:~$
```

Al estar en una sesión de ssh, para que surjan los cambios, debemos de cerrar y volver a entrar.

```
usuario@rrodgarelk:~$ docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS    PORTS    NAMES
usuario@rrodgarelk:~$
```

Listo.

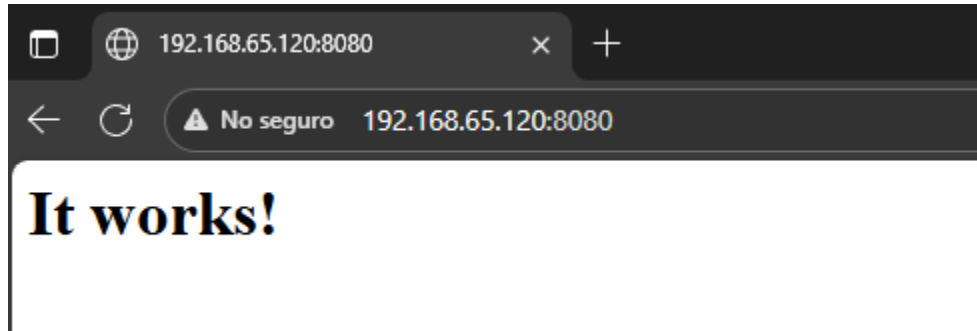
Para comprobar que funciona, voy a levantar un contenedor de apache, el famoso servidor web.

```
usuario@rrodgarelk:~$ docker run -d -p 8080:8080 httpd
Unable to find image 'httpd:latest' locally
latest: Pulling from library/httpd
5b4d5959fc75: Download complete
0e4bc2bd6656: Extracting 1 s
4742a9e996dl: Download complete
4f4fb700ef54: Download complete
87a14f083967: Download complete
9cd0271fa751: Download complete
a675b28a7d22: Download complete
02a2ab6f64cl: Download complete
```

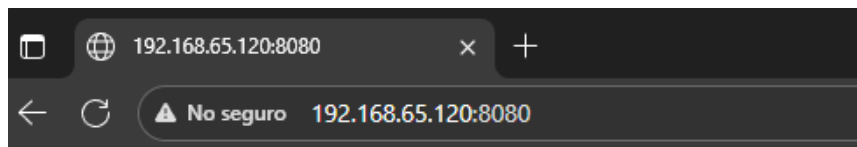
Aquí lo creo, al no tener la imagen se descarga....

```
usuario@rrodgarelk: ~
usuario@rrodgarelk:~$ docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS    PORTS
NAMES
0af77da5c23d   httpd     "httpd-foreground"      47 seconds ago    Up 46 seconds    0.0.0.0:8080->80/tcp, [::]:8080->80
p   sweet_borg
usuario@rrodgarelk:~$
```

Si desde nuestro host, ponemos la ip y el 8080, porque está mapeado al contenedor nos aparecerá el servidor web.



Este es el resultado. Para más confirmación, voy a cambiar el it works, metiéndome dentro del contenedor.



Y aquí lo tenemos cambiado.

Tras esto, vemos que es un software muy potente, ya que no debemos de instalar servicios integrados en la máquina, sino que corren de forma aislada al sistema operativo.

Paso 1º Implantación del ELK.

En este primer paso se procederá a la implantación del stack ELK (Elasticsearch, Logstash y Kibana) sobre Ubuntu Server. Para garantizar una instalación ordenada, reproducible y fácilmente gestionable, se utilizará un fichero compose de docker como herramienta principal de despliegue.

El uso de docker-compose permite:

1. Definir todos los servicios del stack en un único fichero.
2. Especificar puertos, volúmenes y dependencias de manera declarativa.
3. Facilitar la puesta en marcha y el mantenimiento del entorno con comandos simples (, ,).
4. Asegurar que las capturas y la documentación reflejan un proceso estandarizado y replicable.

De esta forma, la implantación del ELK quedará perfectamente estructurada y lista para ser utilizada en los siguientes pasos de la práctica.

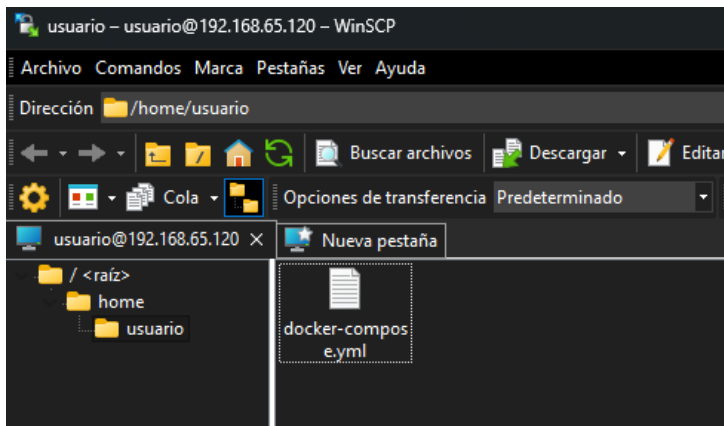
Usaremos un docker-compose, que es un fichero que hace todo lo necesario para levantar y configurar el entorno de trabajo, de forma rápida y sencilla.

```
version: '3.8'
services:
  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.15.0
    container_name: elasticsearch-node
    environment:
      - discovery.type=single-node
      - ES_JAVA_OPTS=-Xms1g -Xmx3g
      - xpack.security.enabled=false
    ports:
      - "9200:9200"
      - "9300:9300"
    volumes:
      - es_data:/usr/share/elasticsearch/data
    networks:
      - es_network

  kibana:
    image: docker.elastic.co/kibana/kibana:8.15.0
    container_name: kibana
    environment:
      - ELASTICSEARCH_HOSTS=http://elasticsearch:9200 # Point Kibana to the Elasticsearch node
    ports:
      - "5601:5601" # Expose Kibana on port 5601
    networks:
      - es_network
    depends_on:
      - elasticsearch # Ensure Elasticsearch starts before Kibana

  logstash:
    image: docker.elastic.co/logstash/logstash:8.15.0
    container_name: logstash
    volumes:
      - logstash_config:/usr/share/logstash/config # Mount the volume with Logstash configuration
      - logstash_pipeline:/usr/share/logstash/pipeline # Mount the volume for pipeline configuration
```

Traspaso el archivo al servidor, mediante scp, con un cliente como WinSCP.



Nos vamos a la consola.

```
usuario@rrodgarelk:~$ docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
usuario@rrodgarelk:~$ ls
docker-compose.yml
usuario@rrodgarelk:~$
```

Tenemos ahí el fichero, ahora con que hagamos un docker compose up o en su defecto -d para que no nos salga todo el proceso...se levantará el entorno.

```
usuario@rrodgarelk:~$ docker compose up -d
WARN[0000] /home/usuario/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 27/34
  -: logstash [#####] 129.4MB / 492.6MB Pulling
    db33482f1e04 Download complete      0.6s
    bbee25ee946c Download complete      4.9s
    26488eb933d4 Download complete      0.9s
    45b1fd29c944 Downloading [=====>] ... 5.4s
    12a8edbceb65 Download complete      0.8s
    7d5f2a507bae Download complete      0.8s
    1ebb4dc57ea8 Download complete      1.3s
    6eecd48a52f Download complete      0.9s
    46ef65f74cf8 Download complete      0.9s
    d0fca5b18401 Download complete      1.1s
  -: elasticsearch [#####] 99.6MB / 668.8MB Pulling
    1fd631a7f77b Download complete      1.4s
    7aaf9e867095 Download complete      0.7s
    8cadcc84c16df Download complete      0.3s
    be0505076ce2 Download complete      0.5s
    4ca545ee6d5d Download complete      0.7s
    dc9db6ea5ff2 Downloading [=====>] ... 5.5s
    dd7fea410473 Download complete      0.4s
    bef9b66d64c1 Extracting 3 s         5.5s
    8c9ellefa7e5 Download complete      1.0s
    8d0f979b52e8 Download complete      0.9s
  -: kibana [###] 88.47MB / 390MB Pulling
    50c27f863681 Download complete      0.9s
    f6b84247827c Download complete      1.6s
    bla5a823635f Downloading [=====>] ... 5.4s
    7ff3c1349574 Download complete      0.9s
    acaf211e68f0 Download complete      1.5s
    e6482380a03e Download complete      0.8s
    5d37849b8bfb Download complete      0.9s
    38a4f0acelb9 Download complete      1.0s
    93469ea36cfe Download complete      0.8s
    40d69daa44b4 Download complete      0.9s
    9bdcdf41232d Download complete      0.6s
```

Tardará un rato.

Se están descargando las imágenes de los contenedores...

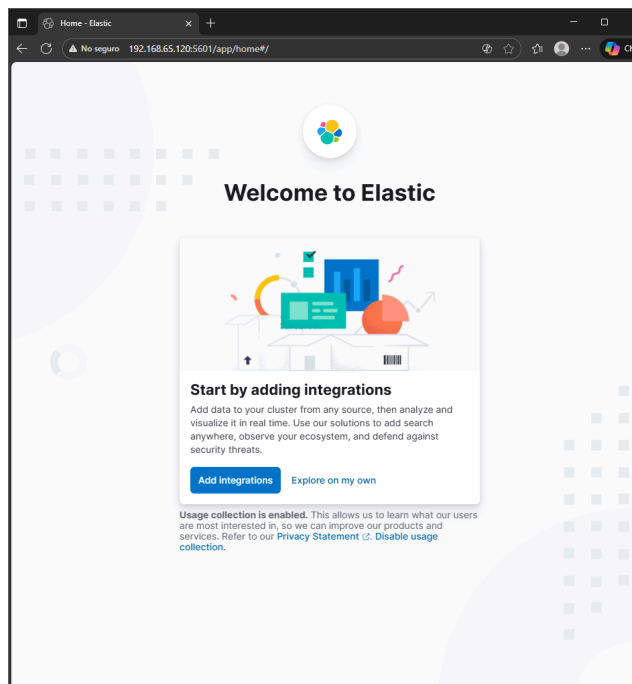
```
[+] Running 7/7
✔ Network usuario_es_network      Created
✔ Volume usuario_es_data          Created
✔ Volume usuario_logstash_config  Created
✔ Volume usuario_logstash_pipeline Created
✔ Container elasticsearch-node     Started
✔ Container logstash               Started
✔ Container kibana                 Started
usuario@rrodgarelk:~$
```

Y en pocos minutos ... ya lo tenemos levantado, comprobamos los estados de los contenedores...

```
usuario@rrodgarelk:~$ docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS
PORTS
45389d2d08f4   docker.elastic.co/logstash/logstash:8.15.0   "/usr/local/bin/dock..."   2 minutes ago   Up 2 m
logstash
58f8e2f1bba1   docker.elastic.co/kibana/kibana:8.15.0       "/bin/tini -- /usr/l..."   2 minutes ago   Up 2 m
kibana
d4db5fa7b3dd   docker.elastic.co/elasticsearch/elasticsearch:8.15.0   "/bin/tini -- /usr/l..."   2 minutes ago   Up 2 m
elasticsearch-node
usuario@rrodgarelk:~$
```

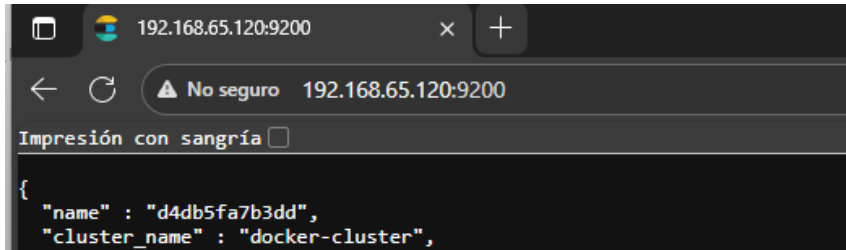
Pues en orden, todo está bien.

Vamos a comprobar que verdaderamente podemos acceder desde nuestro host.



Kibana funciona.

elasticsearch...



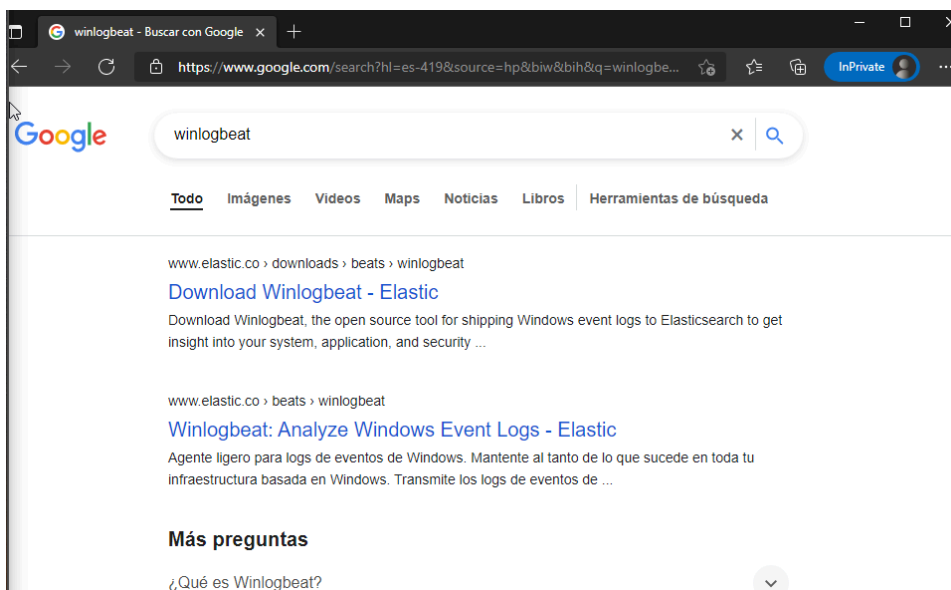
Y logstash, funciona. Es un contenedor que no tiene nada gráfico que diga que haya errores, salvo los logs en consola claro.

Paso 1.2 Configuración de Windows para Kibana (con winlogbeat).

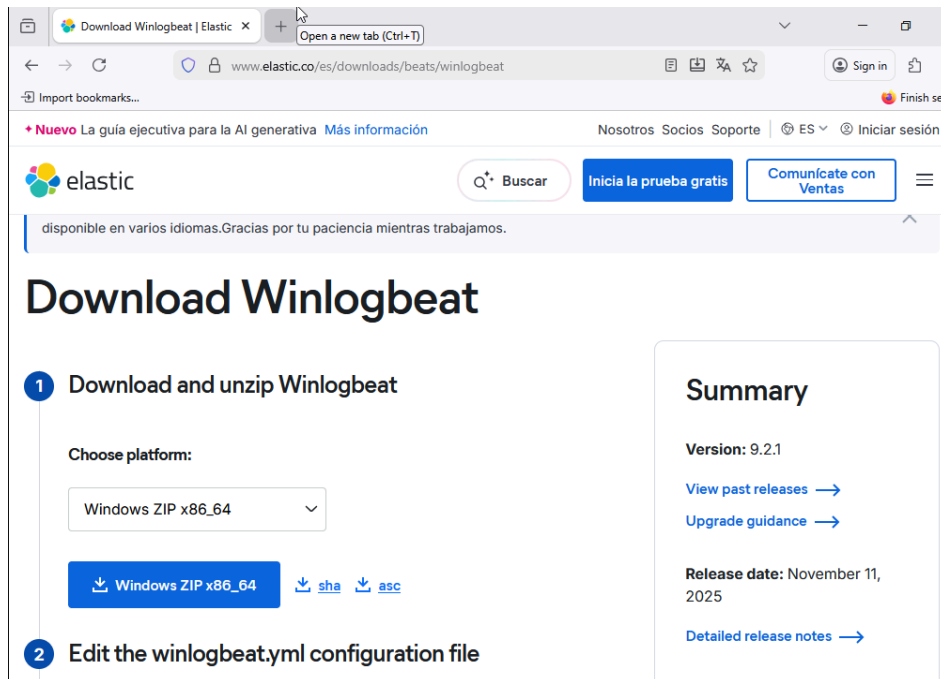
Validar que el stack ELK no solo está levantado, sino que también puede recibir, almacenar y mostrar datos de manera integrada.

Con ello, además de la vm servidor, debemos de tener un cliente, en mi caso usaré un Windows 10 pro, para así tener métricas de forma fácil y sencilla.

Nos vamos al cliente (previamente, se ha tenido que instalar y crear la vm de windows).



Vamos al primer enlace.

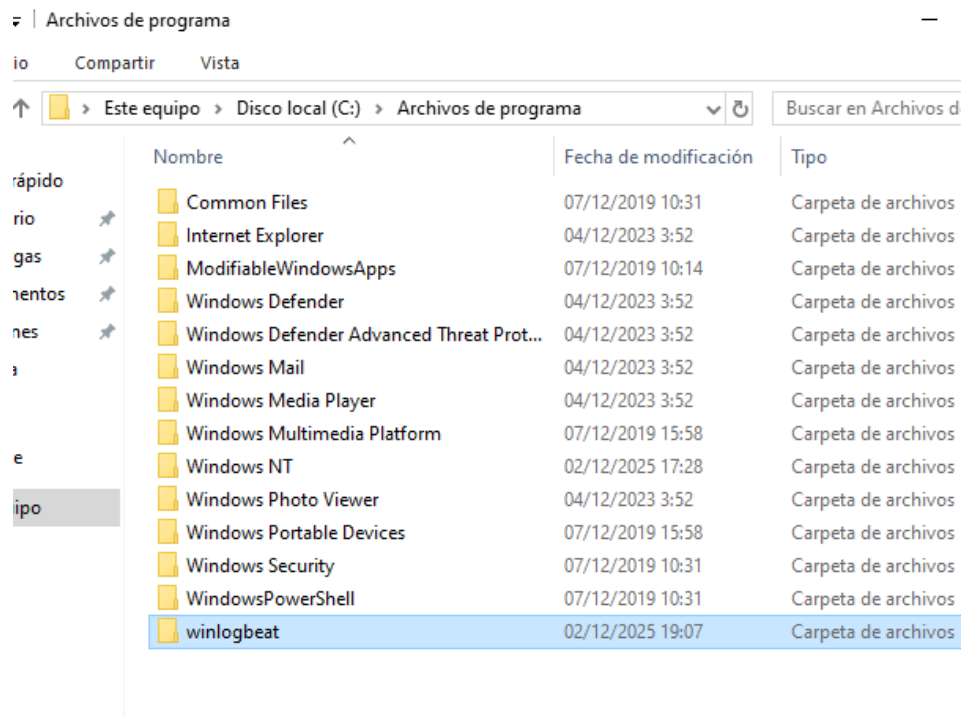


(por temas de actualización, usaré Mozilla).

Dicho ese paréntesis, descargamos el .ZIP, no el msi.

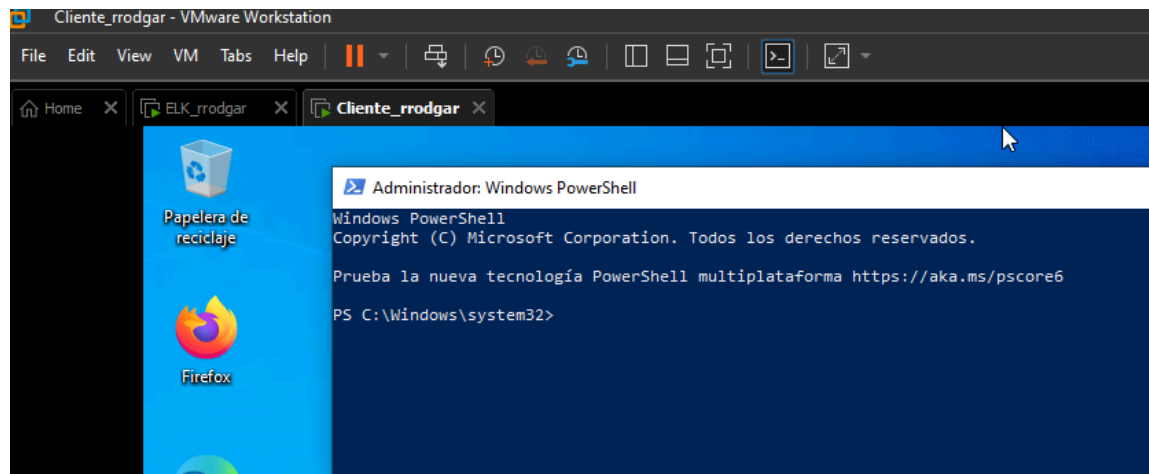
Tras descargarlo, descomprimos...

Teniendo en cuenta, que debemos de poner la carpeta en C:\Archivos de programa.



Paso 1.2.1 Configuración del servicio winlogbeat.

Abrimos una consola de PS, en modo administrador.



```
PS C:\Windows\system32> set-location "C:\Program Files\winlogbeat\  
PS C:\Program Files\winlogbeat> █
```

Una vez dentro, debemos de deshabilitar por un momento la política de restricción de ejecución de script, si no nos funcionará

```
PS C:\Program Files\winlogbeat> Set-ExecutionPolicy -ExecutionPolicy Unrestricted  
  
Cambio de directiva de ejecución  
La directiva de ejecución te ayuda a protegerte de scripts en los que no confías. Si cambias dicha directiva, podrías exponerte a los riesgos de seguridad descritos en el tema de la Ayuda about_Execution_Policies en https://go.microsoft.com/fwlink/?LinkID=135170. ¿Quieres cambiar la directiva de ejecución?  
[S] Sí [O] Sí a todo [N] No [T] No a todo [U] Suspender [?] Ayuda (el valor predeterminado es "N"): O  
PS C:\Program Files\winlogbeat> █
```

Luego, es de obligado cumplimiento, dejarlo de nuevo en restringido, esto es en entorno de pruebas.

```
PS C:\Program Files\winlogbeat> .\install-service-winlogbeat.ps1  
  
Advertencia de seguridad  
Ejecute solo los scripts de confianza. Los scripts procedentes de Internet pueden ser útiles, pero este script podría dañar su equipo. Si confía en este script, use el cmdlet Unblock-File para permitir que se ejecute sin este mensaje de advertencia. ¿Desea ejecutar C:\Program Files\winlogbeat\install-service-winlogbeat.ps1?  
[N] No ejecutar [Z] Ejecutar una vez [U] Suspender [?] Ayuda (el valor predeterminado es "N"): Z  
  
Status      Name      DisplayName  
-----  
Stopped    winlogbeat    winlogbeat
```

Antes de ponerlo en marcha el servicio, debemos de configurar el fichero de winlogbeat.yml del cliente windows.

```
*winlogbeat: Bloc de notas
Archivo Edición Formato Ver Ayuda
#cloud.auth:

# ===== Outputs =====

# Configure what output to use when sending the data collected by the beat.

# ----- Elasticsearch Output -----
output.elasticsearch:
  # Array of hosts to connect to.
  hosts: ["192.168.78.20:9200"]

  # Protocol - either `http` (default) or `https`.
  #protocol: "https"

  # Authentication credentials - either API key or username/password.
  #api_key: "id:api_key"
  #username: "elastic"
  #password: "changeme"

  # Pipeline to route events to security, sysmon, or powershell pipelines.
  pipeline: "winlogbeat-%{agent.version}}-routing"

# ----- Logstash Output -----
#output.logstash:
  # The Logstash hosts
  #hosts: ["192.168.78.20:5044"]

  # Optional SSL. By default is off.
  # List of root certificates for HTTPS server verifications
  #ssl.certificate_authorities: ["/etc/pki/root/ca.pem"]

  # Certificate for SSL client authentication
  #ssl.certificate: "/etc/pki/client/cert.pem"

  # Client Certificate Key
  #ssl.key: "/etc/pki/client/cert.key"

# ===== Processors =====
```

(Tener en cuenta que la vm esté en red).

nota: abrid en una cmd o ps como administrador, y en la ubicación de winlogbeat, un notepad, porque es primordial para editar y modificar.

Usaremos el siguiente comando para crear los dashboards...

```
PS C:\Program Files\winlogbeat> .\winlogbeat.exe setup -c .\winlogbeat.yml -e
```

```
PS C:\Program Files\winlogbeat> .\winlogbeat.exe setup -c .\winlogbeat.yml -e
{"@version": "8", "@timestamp": "2025-12-02T19:20:32.515+0100", "log_origin": {"function": "github.com/elastic/beats/v7/libbeat/cmd/in
stance.(Beat).configure", "file.name": "instance/beat.go", "file.line": 835}, "message": "Home path: [C:\\Program Files\\winlogbeat] Config
path: [C:\\Program Files\\winlogbeat] Data path: [C:\\Program Files\\winlogbeat\\data] Logs path: [C:\\Program Files\\winlogbeat\\logs
]", "service.name": "winlogbeat", "ecs.version": "1.6.0"}
{"@version": "8", "@timestamp": "2025-12-02T19:20:32.523+0100", "log_origin": {"function": "github.com/elastic/beats/v7/libbeat/cmd/in
stance.(Beat).configure", "file.name": "instance/beat.go", "file.line": 843}, "message": "Beat ID: 8f8ae58b-4a03-4d4a-94a4-71346f0554fc", "se
vice.name": "winlogbeat", "ecs.version": "1.6.0"}
{"@version": "8", "@timestamp": "2025-12-02T19:20:32.551+0100", "log_logger": "beats", "log_origin": {"function": "github.com/elastic/beats
/v7/libbeat/cmd/instance.(Beat).createBeater", "file.name": "instance/beat.go", "file.line": 332}, "message": "Setup Beat: winlogbeat; Ver
sion: 9.2.1 (FPS) (Elastic Stack)", "service.name": "winlogbeat", "ecs.version": "1.6.0"}
{"@version": "8", "@timestamp": "2025-12-02T19:20:32.552+0100", "log_logger": "beats", "log_origin": {"function": "github.com/elastic/beats
/v7/libbeat/cmd/instance.(Beat).logSystemInfo", "file.name": "instance/beat.go", "file.line": 1392}, "message": "Beat info", "service.name":
"winlogbeat", "system_info": {"beat": {"path": "config:C:\\Program Files\\winlogbeat", "data": "C:\\Program Files\\winlogbeat\\data", "ho
me": "C:\\Program Files\\winlogbeat", "logs": "C:\\Program Files\\winlogbeat\\logs"}, "type": "winlogbeat", "uid": "8f8ae58b-4a03-4d4a-94a4-
1346f0554fc"}, "ecs.version": "1.6.0"}
{"@version": "8", "@timestamp": "2025-12-02T19:20:32.562+0100", "log_logger": "beats", "log_origin": {"function": "github.com/elastic/beats
/v7/libbeat/cmd/instance.(Beat).logSystemInfo", "file.name": "instance/beat.go", "file.line": 1401}, "message": "Build info", "service.name":
"winlogbeat", "system_info": {"build": {"commit": "6485elled8f854d7792dfc2999bf19db37315ead", "libbeat": "9.2.1", "time": "2025-11-06T08:47:
7.000Z", "version": "9.2.1"}, "ecs.version": "1.6.0"}
{"@version": "8", "@timestamp": "2025-12-02T19:20:32.563+0100", "log_logger": "beats", "log_origin": {"function": "github.com/elastic/beats
/v7/libbeat/cmd/instance.(Beat).logSystemInfo", "file.name": "instance/beat.go", "file.line": 1404}, "message": "Go runtime info", "service
name": "winlogbeat", "system_info": {"go": {"os": "windows", "arch": "amd64", "max_procs": 2, "version": "go1.24.9"}, "ecs.version": "1.6.0"}
{"@version": "8", "@timestamp": "2025-12-02T19:20:32.566+0100", "log_logger": "beats", "log_origin": {"function": "github.com/elastic/beats
/v7/libbeat/cmd/instance.(Beat).logSystemInfo", "file.name": "instance/beat.go", "file.line": 1410}, "message": "Host info", "service.name":
"winlogbeat", "system_info": {"host": {"architecture": "x86_64", "native_architecture": "x86_64", "boot_time": "2025-12-02T17:27:50+0100", "n
ame": "DESKTOP-TP27J51", "ip": [{"fe80:d89a:aad3:7e21:182a", "192.168.78.157"}, {"::1", "127.0.0.1"}], "kernel_version": "10.0.19041.3803 (WinBu
ld.160101.08000)", "mac": [{"00:c0:2d:bca:de:6e:90"}, {"::"}, {"type": "windows", "family": "windows", "platform": "Windows", "name": "Windows 10 Pro", "v
ersion": "10.0", "major": "10", "minor": "0", "patch": "0", "build": "19045.3803"}, "timezone": "CET", "timezone_offset_sec": 3600, "id": "30ff1f3-d0a5-44
0-a0e0-32f5bd423830"}, "ecs.version": "1.6.0"}
{"@version": "8", "@timestamp": "2025-12-02T19:20:32.567+0100", "log_logger": "beats", "log_origin": {"function": "github.com/elastic/beats
/v7/libbeat/cmd/instance.(Beat).logSystemInfo", "file.name": "instance/beat.go", "file.line": 1439}, "message": "Process info", "service.na
```

Y luego iniciamos el servicio.

Start-Service winlogbeat.

Paso 2º Visualización y análisis de eventos en Kibana.

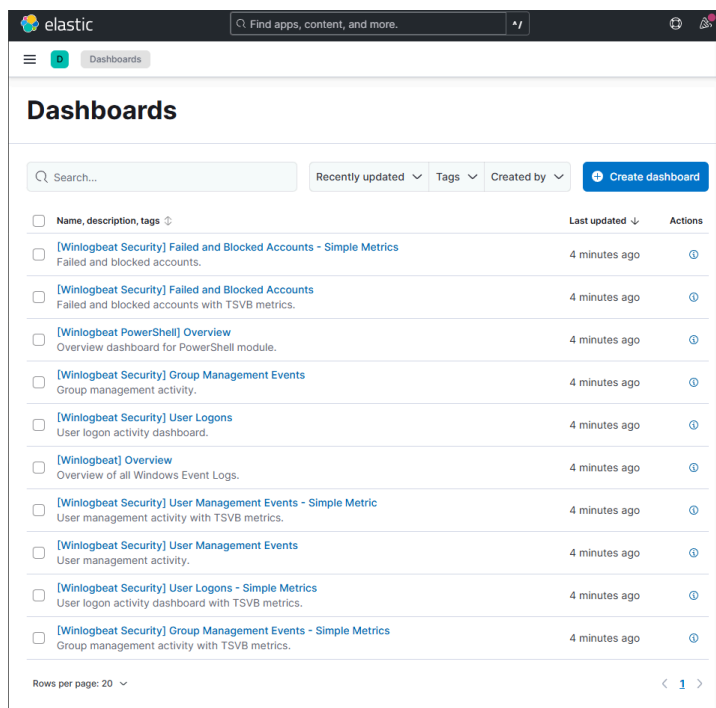
Una vez el stack ELK se encuentra operativo y el cliente Windows envía correctamente los eventos mediante Winlogbeat, se procede a la visualización y análisis de los datos en Kibana.

Desde la sección de dashboards, Kibana nos ofrece un conjunto de dashboard preconfigurados, que podemos verlos y consultarlos.

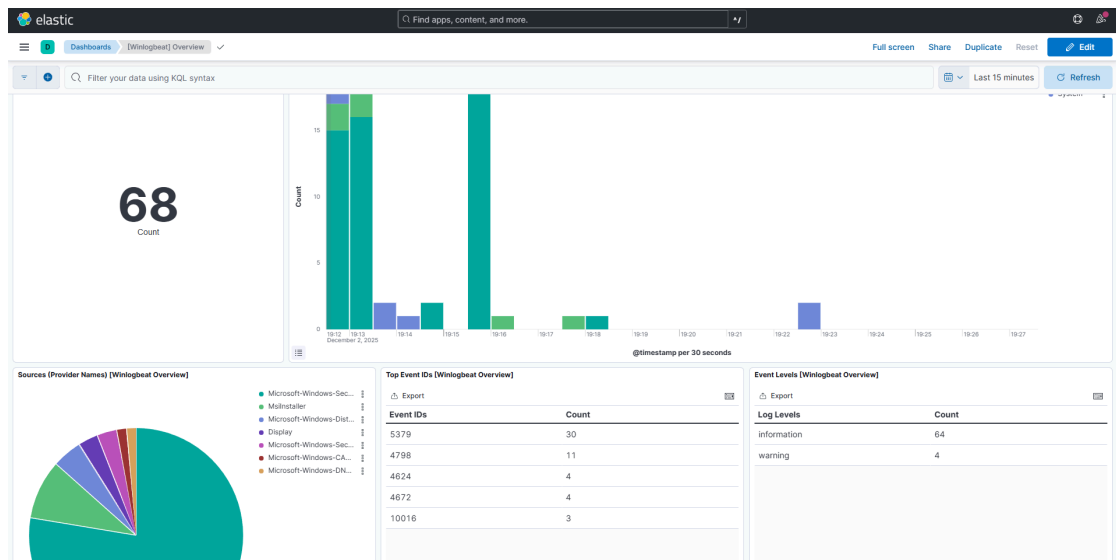
Los registros recopilados incluyen, entre otros:

- Eventos de inicio y cierre de sesión de usuarios
- Intentos de autenticación fallidos
- Creación y modificación de cuentas
- Eventos de seguridad del sistema
- Actividad general del sistema operativo

Esta visualización centralizada permite analizar de forma rápida y eficaz la actividad del equipo monitorizado, constituyendo la base para la detección de incidentes de ciberseguridad.

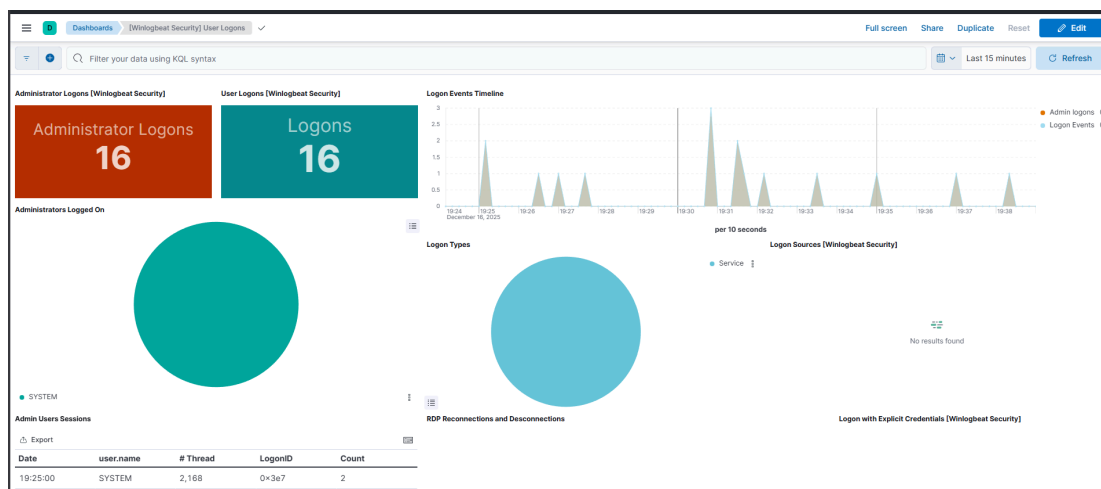


Tenemos todos los dashboard, que se importaron previamente al logstash, y ahora están aquí por secciones, si clicamos en por ejemplo Winlogbeat Overview....nos aparecerá esto...

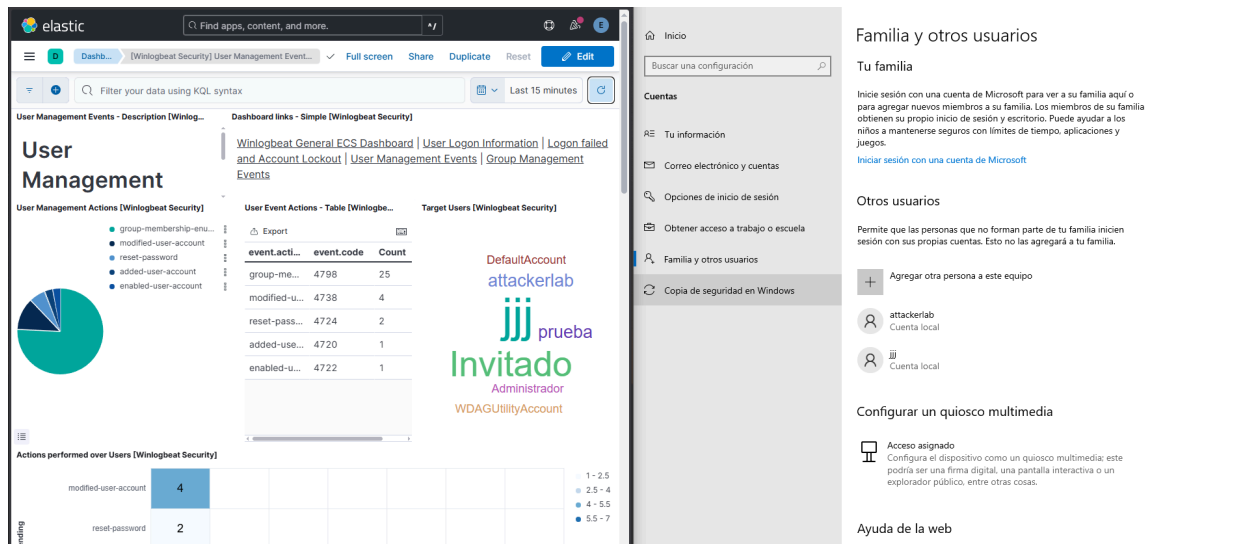


Tenemos aquí el gráfico del cliente, un resumen general.

Podemos navegar por cada una, y veremos el potencial que tiene esto.



Esto es por ejemplo los user logon, aquí podemos intentar que se registre lo del sistema que tengamos enrolado.



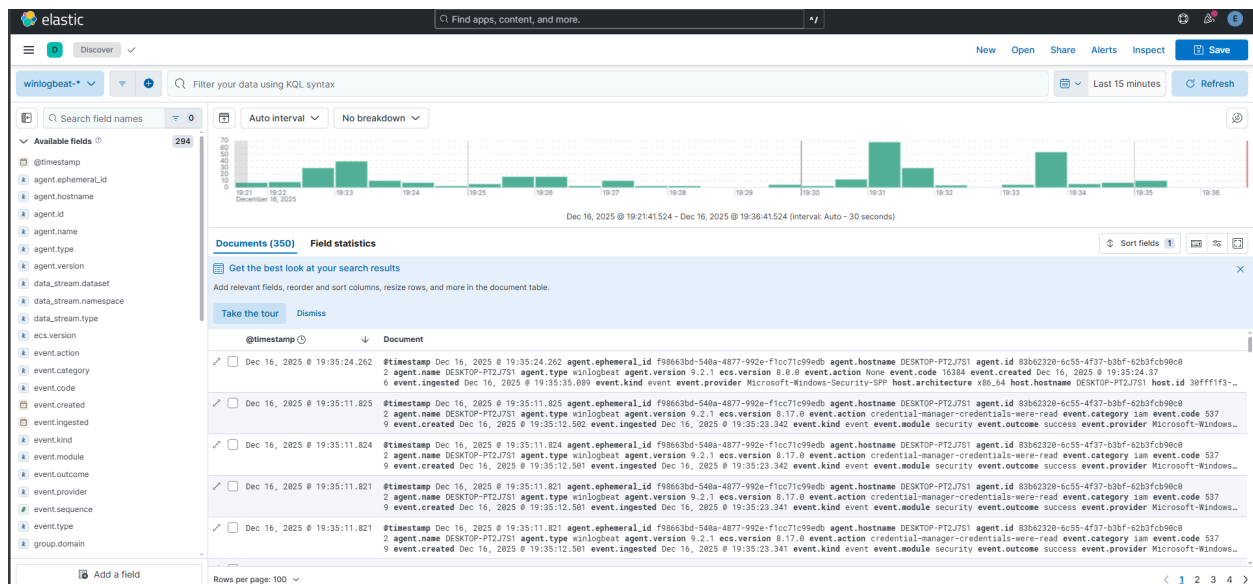
Captura del apartado de eventos de usuario..

Paso 2.1 Discover.

El apartado Discover de Kibana, permite explorar de forma detallada los eventos de los equipos enrolados.

Desde esta sección es posible:

- Visualizar todos los eventos generados por los sistemas monitorizados
- Aplicar filtros temporales y por campos específicos
- Analizar el contenido completo de cada registro
- Identificar patrones sospechosos o comportamientos anómalos



Este panel nos muestra el flujo de los logs en tiempo real de los equipos.

En el contexto de esta práctica, Discover se utiliza principalmente para el análisis de eventos de seguridad de Windows enviados por Winlogbeat, como intentos de inicio de sesión, autenticaciones fallidas y cambios en cuentas de usuario.

Discover permite filtrar eventos utilizando campos específicos como `event.code`, `event.action` o `winlog.event\_id`, facilitando la identificación precisa de eventos relevantes para la seguridad del sistema.

Paso 2.2 HTTPS en la comunicación de métricas.

*Con el script, se crean los certificados autofirmados, para la causa de este laboratorio.

(El script se subirá dentro de unos meses, está mejorandose la implementación).

En la implementación inicial del laboratorio, la transmisión de métricas desde el cliente Windows al stack ELK se realizaba mediante HTTP. Para aumentar la seguridad y proteger la información, se ha configurado el flujo de datos para que viaje mediante HTTPS.

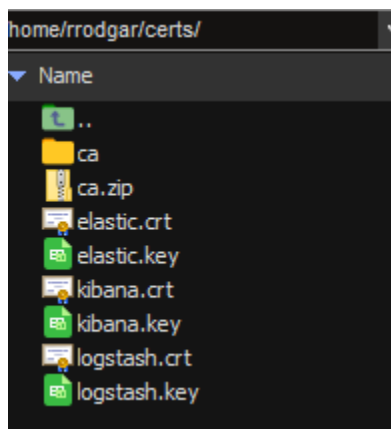
Para facilitar esta configuración, se ha desarrollado un ****script automatizado**** que realiza las siguientes acciones:

- Genera certificados autofirmados para Elasticsearch y Kibana
- Configura el stack ELK para habilitar la comunicación cifrada mediante HTTPS
- Ajusta la configuración de Winlogbeat para que utilice los certificados generados

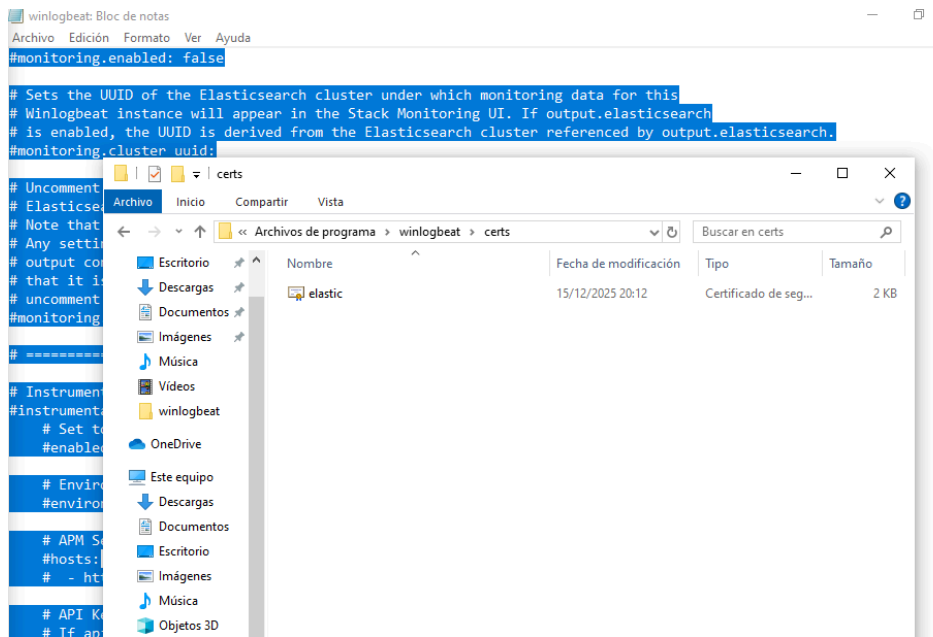
Al tratarse de certificados autofirmados, el cliente Windows inicialmente no confía en ellos. Por este motivo, es necesario descargar el certificado generado en el servidor y copiarlo al cliente, dentro de una carpeta denominada `certs` en la instalación de Winlogbeat. En el fichero de configuración `winlogbeat.yml`, se especifica el uso de este certificado para establecer la comunicación cifrada con Elasticsearch y Kibana.

Gracias a este procedimiento, todas las métricas y logs enviados desde el cliente Windows viajan de forma segura, evitando la interceptación o manipulación de los datos durante su transmisión.

Para ello, tenemos que descargar el certificado de nuestro servidor que usa el ELK, y pasarlo al windows cliente.



Lo pasamos a la vm cliente, y posteriormente, ponemos una carpeta en winlogbeat llamada certs y dentro el certificado.



En el fichero, es necesario, que especifiquemos, el certificado de la imagen anterior, pero especificando en el winlogbeat.yaml.

```

# ----- Elasticsearch Output -----
output.elasticsearch:
  # Array of hosts to connect to.
  hosts: ["192.168.78.20:9200"]

  # Protocol - either `http` (default) or `https`.
  protocol: "https"

  # Authentication credentials - either API key or username/password.
  #api_key: "id:api_key"
  username: "elastic"
  password: "${ELASTIC_PASSWORD}"

  # Pipeline to route events to security, sysmon, or powershell pipelines.
  pipeline: "winlogbeat-%[agent.version]}-routing"

  ssl.certificate_authorities: ["C:\\Program Files\\winlogbeat\\certs\\elastic.crt"]

```

Debemos de tenerlo así, tanto en elasticsearch y kibana.

*Kibana.

```
# ===== Kibana =====  
  
# Starting with Beats version 6.0.0, the dashboards are loaded via the Kibana API.  
# This requires a Kibana endpoint configuration.  
setup.kibana:  
  
  # Kibana Host  
  # Scheme and port can be left out and will be set to the default (http and 5601)  
  # In case you specify an additional path, the scheme is required: http://localhost:5601/path  
  # IPv6 addresses should always be defined as: https://[2001:db8::1]:5601  
  host: "https://192.168.78.128:5601"  
  # CONFIGURACIÓN AÑADIDA/CORREGIDA PARA CONFIAR EN EL CERTIFICADO:  
  ssl.certificate_authorities: ["C:\\Program Files\\winlogbeat\\certs\\Kibana.crt"]  
  
  # Kibana Space ID  
  # ID of the Kibana Space into which the dashboards should be loaded. By default,  
  # the Default Space will be used.  
  #space.id:
```

Todo esto en el fichero de winlogbeat del cliente windows.

Paso 2.3 Gestión segura de credenciales en winlogbeat.

Para asegurar que las credenciales utilizadas por Winlogbeat no se expongan en texto plano, se crea un **keystore** que permite almacenar la contraseña de Elasticsearch de forma cifrada y ofuscada.

Vamos a crear un keystore.

```
PS C:\Program Files\winlogbeat> cd "C:\Program Files\winlogbeat"  
PS C:\Program Files\winlogbeat> .\winlogbeat.exe keystore create  
Created winlogbeat keystore  
PS C:\Program Files\winlogbeat>
```

Añadimos la variable de elastic_password.

```
PS C:\Program Files\winlogbeat> .\winlogbeat.exe keystore add ELASTIC_PASSWORD  
Enter value for ELASTIC_PASSWORD:  
Successfully updated the keystore  
PS C:\Program Files\winlogbeat>
```

Esto lo hacemos, para que en el fichero winlogbeat.yml, podemos poner esto...

“\${elastic_password}”, esto está ligado al keystore, porque en el keystore, tiene la contraseña que usará el comando del siguiente paso para poder enviar las métricas.

Tras correr el comando que se ve, se mandarán los parámetros para enviarlo a Kibana.

```
PS C:\Program Files\winlogbeat> .\winlogbeat.exe setup -e .\winlogbeat.yml -e
{"log.level":"info","@timestamp":"2025-12-15T15:42:29.975+0100","log.origin":{"function":"github.com/elastic/beats/v7/libbeat/cmd/instance.(*Beat).configure","file.name":"instance/beat.go","file.line":835},"message":"Home path: [C:\\Program Files\\winlogbeat] Config path: [C:\\Program Files\\winlogbeat] Data path: [C:\\Program Files\\winlogbeat\\data] Logs path: [C:\\Program Files\\winlogbeat\\logs]","service.name":"winlogbeat","ecs.version":"1.6.0"}
{"log.level":"info","@timestamp":"2025-12-15T15:42:29.985+0100","log.origin":{"function":"github.com/elastic/beats/v7/libbeat/cmd/instance.(*Beat).configure","file.name":"instance/beat.go","file.line":843},"message":"Beat ID: 8f8ae58b-4a03-4d4a-94a4-71346fb554fc","service.name":"winlogbeat","ecs.version":"1.6.0"}
{"log.level":"info","@timestamp":"2025-12-15T15:42:30.138+0100","log.logger":"beat","log.origin":{"function":"github.com/elastic/beats/v7/libbeat/cmd/instance.(*Beat).createBeater","file.name":"instance/beat.go","file.line":1332},"message":"Setup Beat: winlogbeat; Version: 9.2.1 (FIPS-distribution: false)","service.name":"winlogbeat","ecs.version":"1.6.0"}
{"log.level":"info","@timestamp":"2025-12-15T15:42:30.139+0100","log.logger":"beat","log.origin":{"function":"github.com/elastic/beats/v7/libbeat/cmd/instance.(*Beat).logSystemInfo","file.name":"instance/beat.go","file.line":1392},"message":"Beat info","service.name":"winlogbeat","system_info":{"beat":{"path":{"config":"C:\\Program Files\\winlogbeat","data":"C:\\Program Files\\winlogbeat\\data","home":"C:\\Program Files\\winlogbeat","logs":"C:\\Program Files\\winlogbeat\\logs"},"type":"winlogbeat","uuid":"8f8ae58b-4a03-4d4a-94a4-71346fb554fc"},"ecs.version":"1.6.0"}}}
{"log.level":"info","@timestamp":"2025-12-15T15:42:30.144+0100","log.logger":"beat","log.origin":{"function":"github.com/elastic/beats/v7/libbeat/cmd/instance.(*Beat).logSystemInfo","file.name":"instance/beat.go","file.line":1401},"message":"Build info","service.name":"winlogbeat","system_info":{"build":{"commit":"6485e1ledf8854d7792dfc2999bf19db37315ea4"},"libbeat":{"9.2.1","time":"2025-11-06T08:47:37.000Z","version":"9.2.1"},"ecs.version":"1.6.0"}}}
{"log.level":"info","@timestamp":"2025-12-15T15:42:30.145+0100","log.logger":"beat","log.origin":{"function":"github.com/elastic/beats/v7/libbeat/cmd/instance.(*Beat).logSystemInfo","file.name":"instance/beat.go","file.line":1404},"message":"Go runtime info","service.name":"winlogbeat","system_info":{"go":{"os":"windows","arch":"amd64","max_procs":2,"version":"go1.24.9"},"ecs.version":"1.6.0"}}
```

Y tras ello, esperamos a que se envíen los logs al ELK.

Si todo va bien...aparecerá este mensaje:

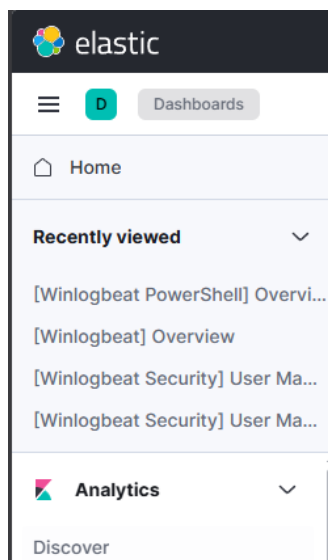
```
Loaded Ingest pipelines
PS C:\Program Files\winlogbeat> █
```

Este mensaje indica que los pipelines de ingestión se han cargado correctamente y que la información se está transmitiendo de forma segura mediante HTTPS.

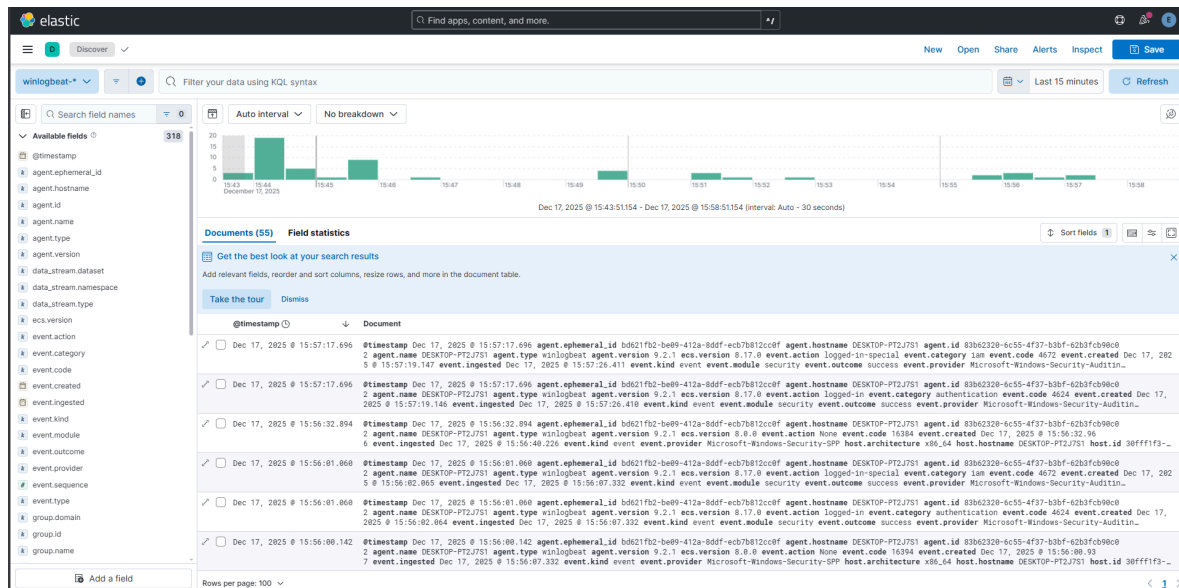
Paso 3º Configuración de alertas en Kibana.

Una de las funcionalidades avanzadas de Kibana es la creación de **alertas automáticas** sobre eventos de Elasticsearch, permitiendo detectar incidentes en tiempo real. En esta práctica, se configura una alerta para **intentos de inicio de sesión fallidos repetidos** (event.code: 4625), típico de ataques de fuerza bruta en RDP.

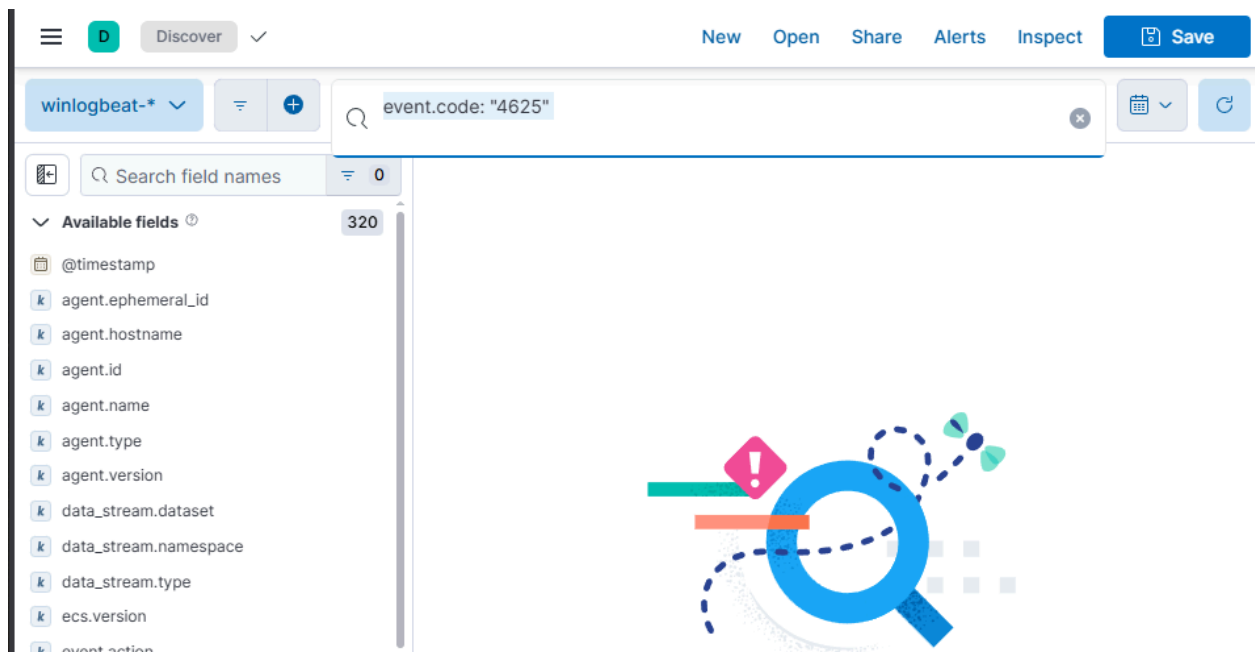
Cuando se cumple, Kibana genera la alerta, que puede registrar el evento, notificar o integrarse con otros sistemas. Esto convierte al stack ELK en un sistema de **detección temprana de incidentes**, además de una herramienta de monitorización.



Accedemos a Discover.

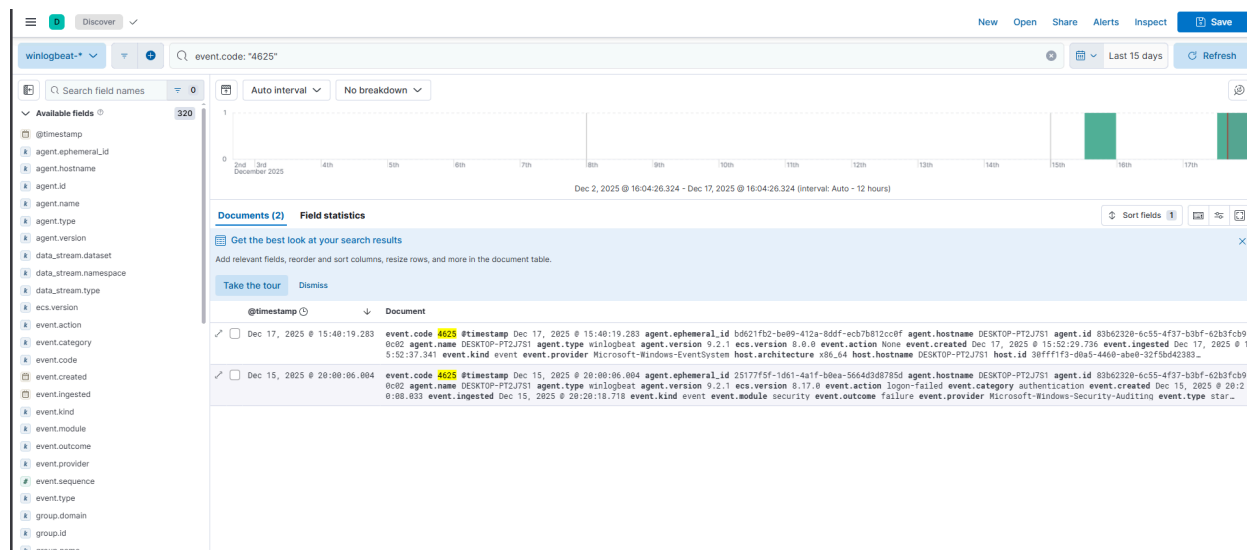


Hacemos una consulta por KQL en la barra de búsqueda de discover.



La alerta se basa en una consulta KQL sobre el índice `winlogbeat-*`, filtrando los eventos de seguridad con `event.code: "4625"`, correspondientes a intentos de inicio de sesión fallidos en sistemas Windows.

Si no aparece registros, como en la captura, ampliamos el rango a últimos 15 días por ejemplo.



Aquí tenemos dos registros.

Paso 3.1 Creación de alertas de seguridad en Kibana (detección de fuerza bruta)

Las técnicas de fuerza bruta y simulación de ataques descritas en esta práctica **se han realizado exclusivamente en un entorno controlado**, compuesto por máquinas virtuales propias y una red aislada creada con fines **educativos y de aprendizaje en ciberseguridad**.

⚠ Est  terminantemente prohibido aplicar estas t cnicas contra sistemas, redes o servicios de terceros sin autorizaci n expresa.

El uso de estas herramientas o m todos fuera de un laboratorio propio puede constituir un delito conforme a la legislaci n vigente en materia de seguridad inform tica y protecci n de sistemas.

El autor **no se hace responsable** del uso indebido de la informaci n aqu  expuesta. Esta documentaci n tiene como  nico objetivo:

- Formaci n t cnica en detecci n de incidentes
- Pr ctica de defensa y monitorizaci n (Blue Team / SOC)
- Comprensi n del funcionamiento de ataques para mejorar la seguridad

El lector asume toda la responsabilidad del uso que haga de esta informaci n.

Para configurar alertas en Kibana se accede a la sección **Stack Management → Rules and Connectors**.

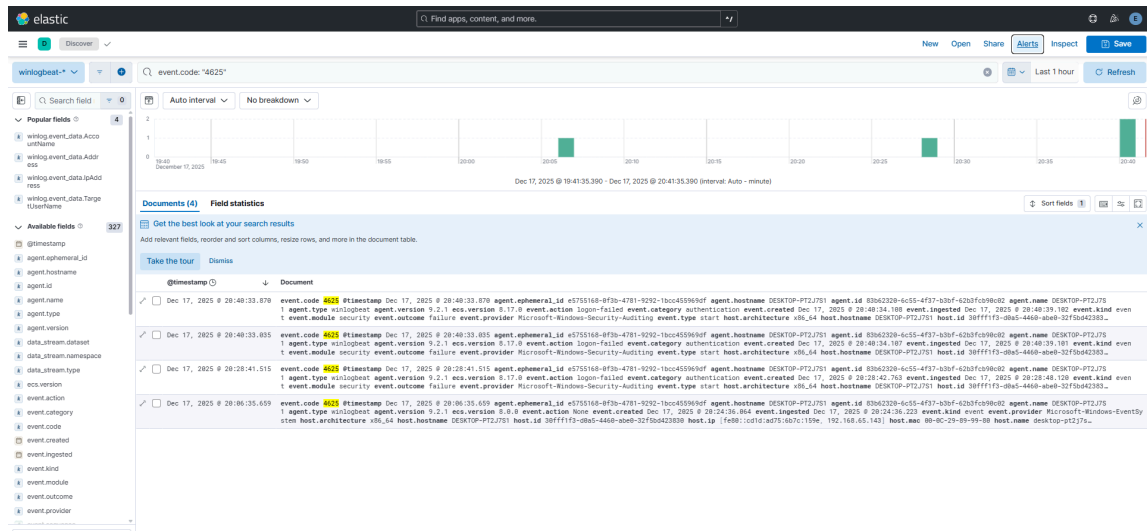
Aquí se pueden crear nuevas reglas basadas en consultas a Elasticsearch y definir acciones automáticas cuando se cumplan ciertas condiciones sobre los eventos recopilados.

Esta sección es el **centro de control de alertas**, donde se gestiona la detección temprana de incidentes y la respuesta automática ante eventos de seguridad.

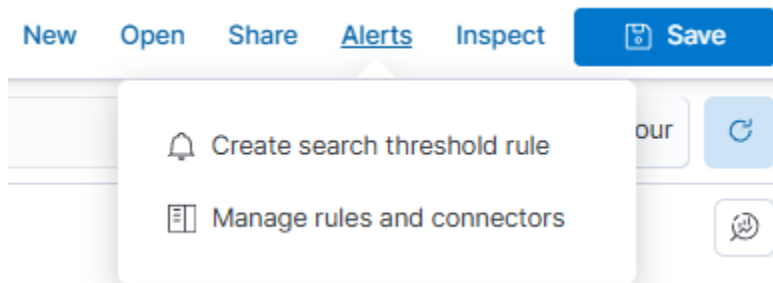
Nos vamos a discover:

buscamos en la barra, de KQL.

Por event.code: "4625"



Y ahora damos en alerts, entre share y inspect.



Create search rule.

Create rule

Name

Tags

Optional

Elasticsearch query

Alert when matches are found during the latest query run. [Learn more](#)

Select a data view

DATA VIEW winlogbeat-*

Define your query

event.code: "4625"

Set the group, threshold, and time window

WHEN count()

OVER all documents

IS ABOVE 1000

FOR THE 5 minutes LAST

Set the number of documents to send

SIZE 100

☐ Exclude matches from previous runs

Cancel

Show API request

✓ Save

Configuramos la regla.

Edit rule ×

Tags Optional

Elasticsearch query
Alert when matches are found during the latest query run. [Learn more](#) ↗

KQL or Lucene ×
Use KQL or Lucene to define a text-based query.

Select a data view
DATA VIEW winlogbeat-*

Define your query

≡ +

event.code: "4625"
AND winlog.logon.type: "Network"
AND NOT source.ip: ("127.0.0.1" OR ":::1")S|

×

Set the group, t

WHEN count()

GROUPED OVER top 1 'winlog.event_data.IpAddress'

IS ABOVE 5

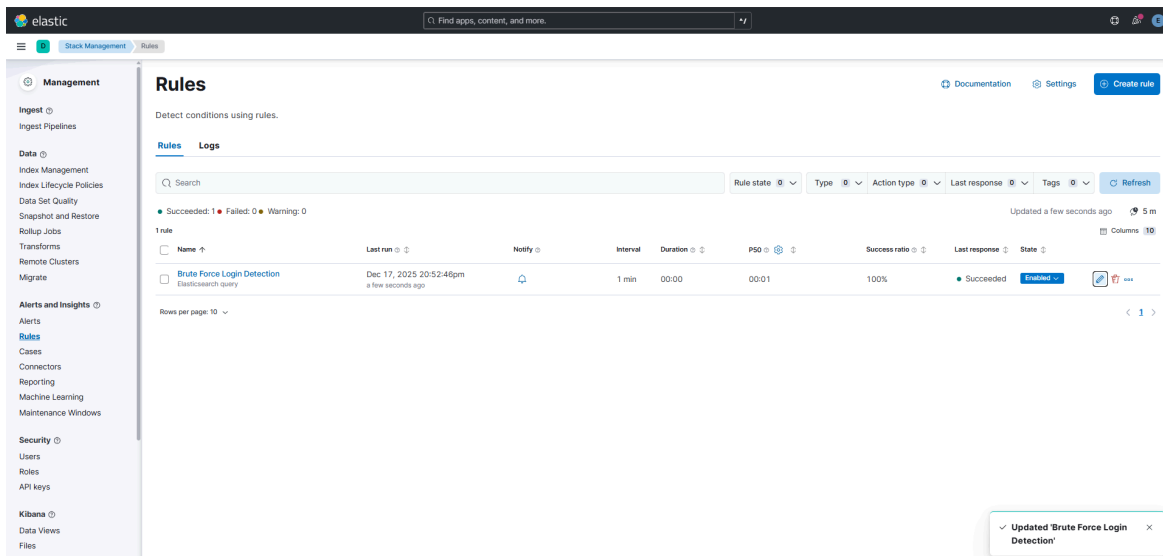
FOR THE LAST 5 minutes

Set the number of documents to send ?

SIZE 100

Buscamos todos los eventos de fallo de inicio de sesión (event.code: 4625) por red (LogonType: 3) excluyendo los intentos que vienen de la máquina local (127.0.0.1 o ::1).

Guardamos.



Aquí presenciamos la nueva regla.

En el próximo paso vamos a probar si funciona la regla verdaderamente.

Paso 3.1.1 Comprobación de la efectividad de la regla de detención.

En este paso, vamos a comprobar si la anterior regla creada en el apartado, funciona o no.

Lo primero, para poder facilitar esto, debemos de desactivar el firewall de windows (solo para probar en laboratorio, **NUNCA** en producción).

```
inet 192.168.65.131/24 brd 192.168.65.255 scope global dynamic noprefixroute eth0
    valid_lft 1167sec preferred_lft 1167sec
inet6 fe80::7abb:483f:af1e:94d8/64 scope link noprefixroute
    valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:60:ed:00:cf brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever

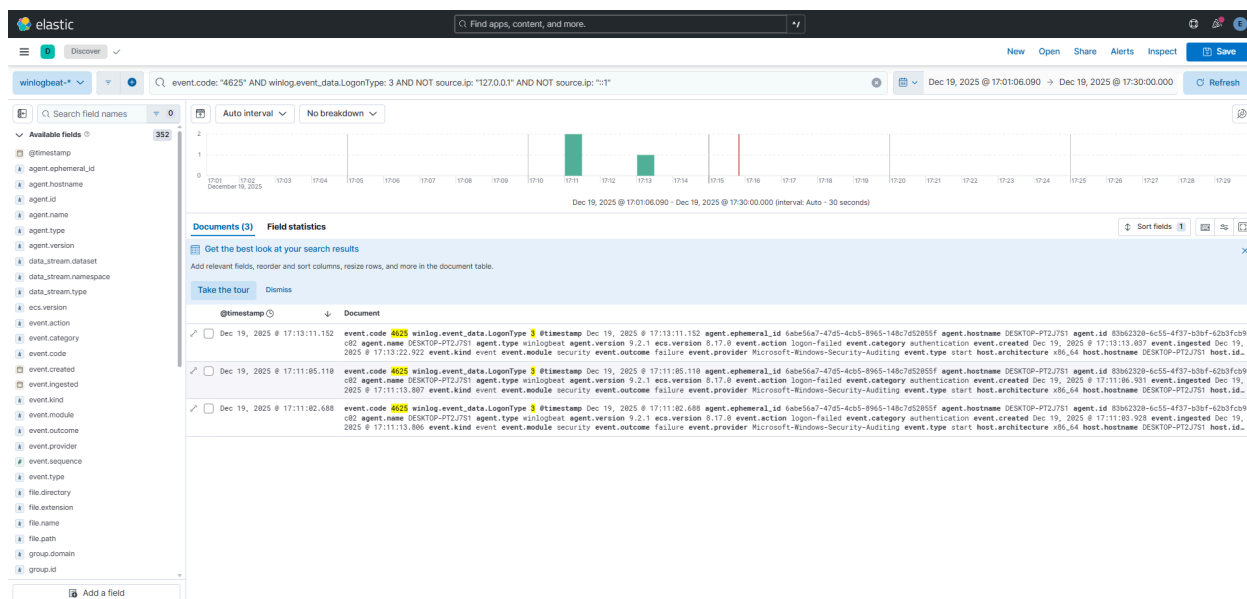
(ruben@rrg)-[~]
$ smbclient -L //192.168.65.143 -U fakeuser

Password for [WORKGROUP\fakeuser]:
session setup failed: NT_STATUS_LOGON_FAILURE

(ruben@rrg)-[~]
$
```

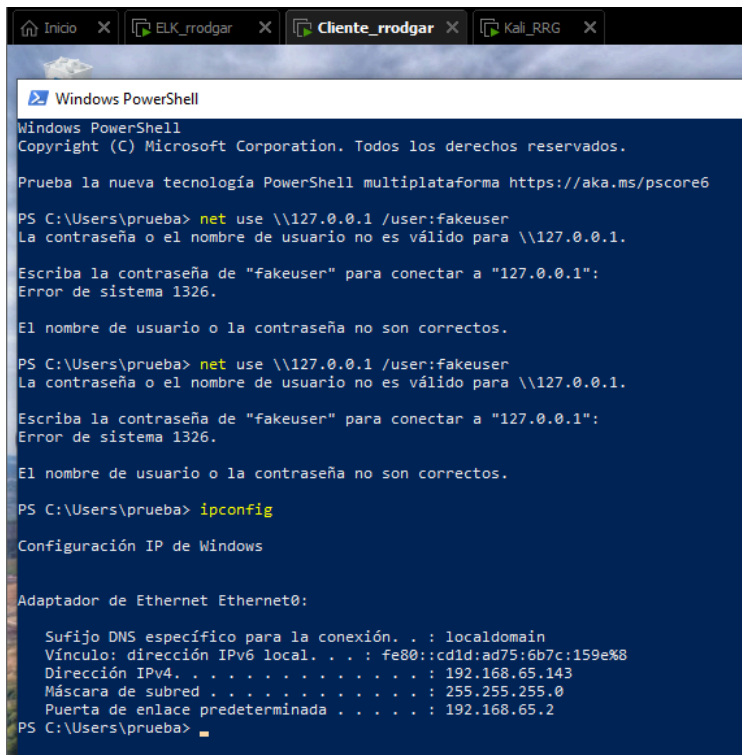
Esta es la máquina Kali, en el comando de smbclient, voy a hacer un login fail, para que el ELK con la regla que ya he comentado, pueda detectar dicha incidencia....

Pues al poner una contraseña que no es correcta....pasará esto en ELK.



Si miramos los detalles...veremos lo siguiente.

```
host.os.family: [
  "19045.6456"
],
"host.ip": [
  "fe80::cd1d:ad75:6b7c:159e",
  "192.168.65.143"
]
```



```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Prueba la nueva tecnología PowerShell multiplataforma https://aka.ms/pscore6

PS C:\Users\prueba> net use \\127.0.0.1 /user:fakeuser
La contraseña o el nombre de usuario no es válido para \\127.0.0.1.

Escriba la contraseña de "fakeuser" para conectar a "127.0.0.1":
Error de sistema 1326.

El nombre de usuario o la contraseña no son correctos.

PS C:\Users\prueba> net use \\127.0.0.1 /user:fakeuser
La contraseña o el nombre de usuario no es válido para \\127.0.0.1.

Escriba la contraseña de "fakeuser" para conectar a "127.0.0.1":
Error de sistema 1326.

El nombre de usuario o la contraseña no son correctos.

PS C:\Users\prueba> ipconfig

Configuración IP de Windows

Adaptador de Ethernet Ethernet0:

    Sufixo DNS específico para la conexión. . . : localdomain
    Vínculo: dirección IPv6 local. . . : fe80::cd1d:ad75:6b7c:159e%8
    Dirección IPv4. . . . . : 192.168.65.143
    Máscara de subred . . . . . : 255.255.255.0
    Puerta de enlace predeterminada . . . . . : 192.168.65.2
PS C:\Users\prueba>
```

La ip del Windows.

Y ahora...

```
{
  "security":
  ],
  "winlog.event_data.SubjectLogonId": [
    "0x0"
  ],
  "related.ip": [
    "192.168.65.131"
  ]
}
```

La ip de Kali Linux.

Con esta regla, hemos podido pillar un incidente de ciberseguridad, a nivel de seguridad claro.

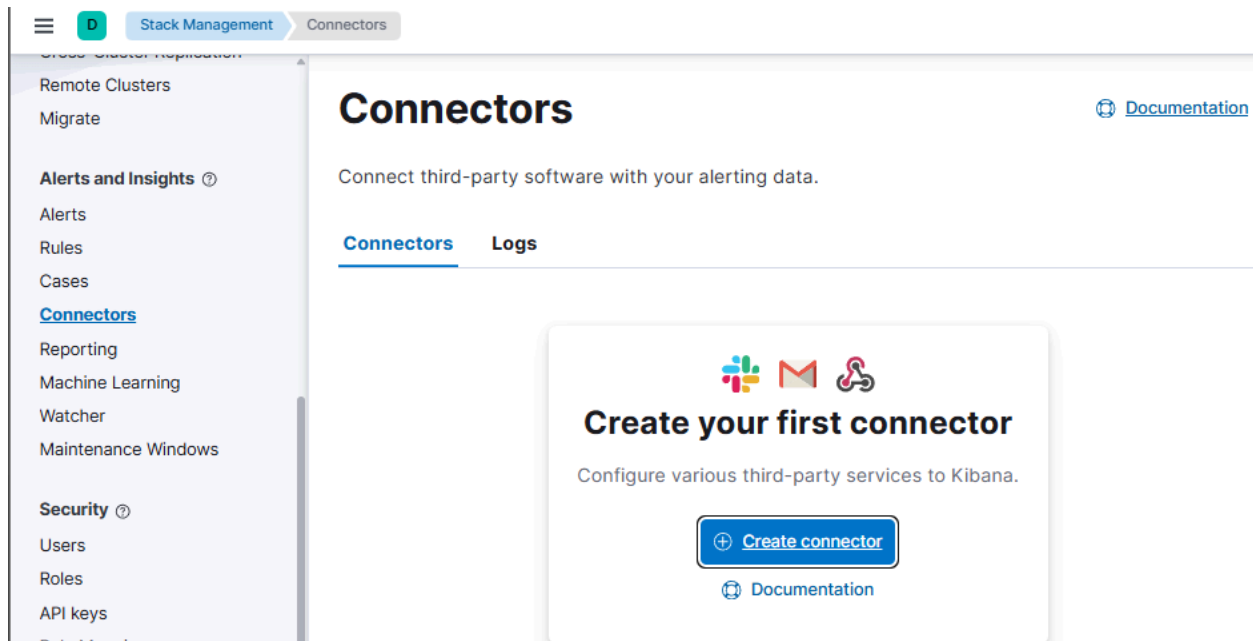
Paso 3.2 Mandar alerta vía Correo.

En este paso se configura el envío de alertas por correo electrónico desde la interfaz web de Kibana, de forma que el administrador reciba una notificación inmediata cuando se detecten intentos de inicio de sesión fallidos que puedan indicar un ataque de fuerza bruta.

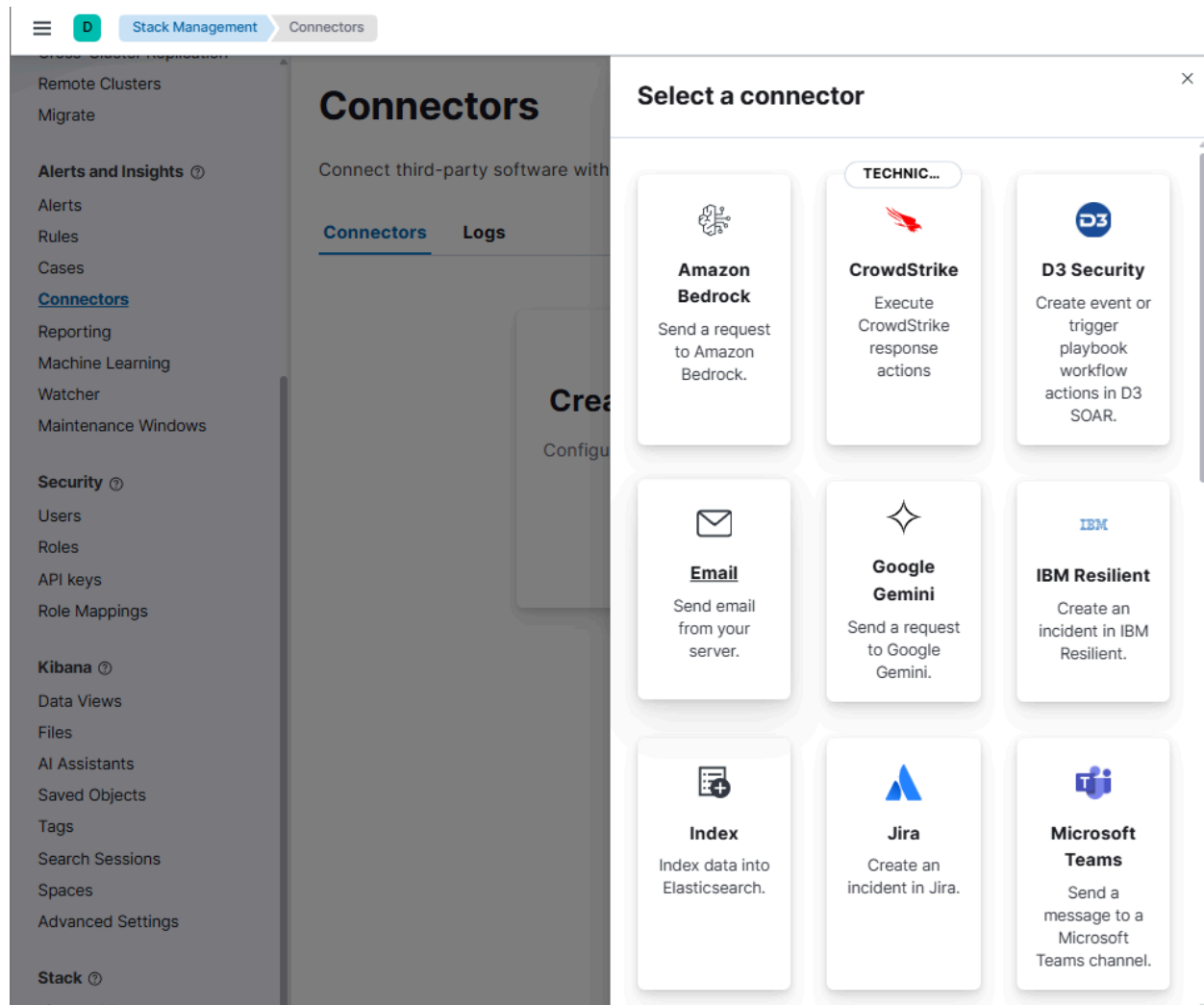
Para ello, nos vamos a stack management...

Connectors.

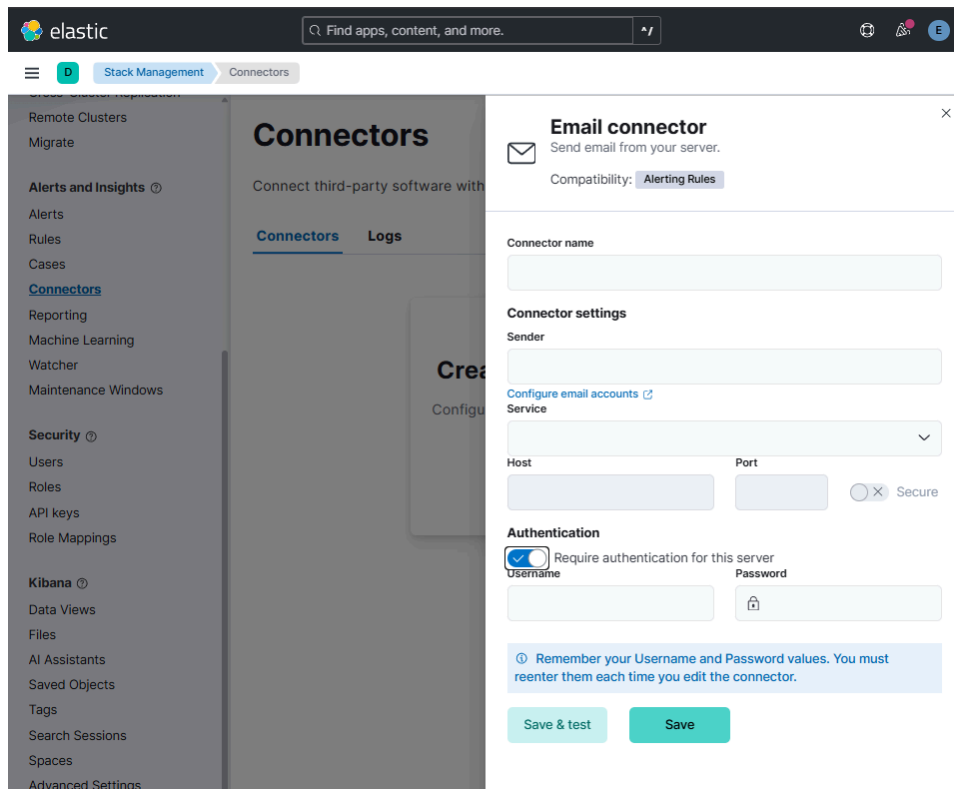
Nota: Para usar connector, necesitamos activar una prueba, ya que es necesario pagar una licencia para poder usarlo, pero nos servirá con la prueba de 30 días.



Damos en Create connector.



Hay infinidad de conectores, incluso con Gemini (IA), pero elegiré Email, ya que es el propósito de este apartado.



The screenshot shows the Elastic Stack Management interface. The left sidebar contains navigation links for Remote Clusters, Alerts and Insights, Security, and Kibana. The main content area is titled 'Connectors' and shows the configuration for an 'Email connector'. The configuration fields include:

- Connector name: A text input field.
- Connector settings: A section containing:
 - Sender: A text input field.
 - Service: A dropdown menu.
 - Host: A text input field.
 - Port: A text input field.
 - Secure: A toggle switch.
- Authentication: A section containing:
 - Require authentication for this server: A toggle switch.
 - Username: A text input field.
 - Password: A text input field with a lock icon.

A note at the bottom states: "Remember your Username and Password values. You must reenter them each time you edit the connector." Below the fields are two buttons: 'Save & test' and 'Save'.

Aquí tenemos que poner un nombre de Connector name.

Yo pondré Prueba-Correo

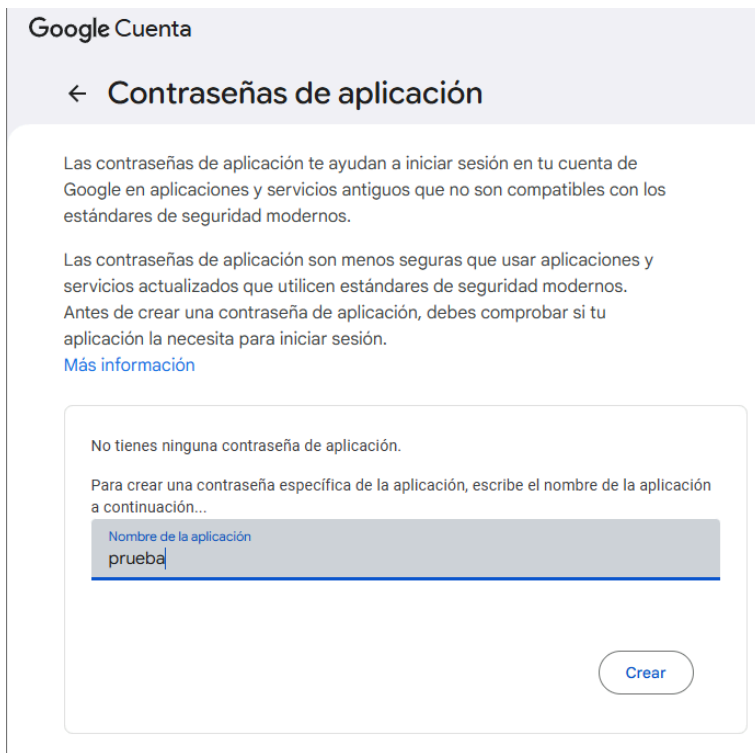
Lo siguiente, me guiaré del link que viene abajo de “configure email account”

[Guia](#)

IMPORTANTE: Hay que hacer un paso intermedio, y además, yo para este paso usé una cuenta educativa, y no es posible, hay que usar una que tengamos.

[Crear app](#) este enlace es importante, tenemos que tener una contraseña para kibana.

Aquí creo una aplicación, para obtener el código que usaremos en el conector de Kibana



Google Cuenta

← Contraseñas de aplicación

Las contraseñas de aplicación te ayudan a iniciar sesión en tu cuenta de Google en aplicaciones y servicios antiguos que no son compatibles con los estándares de seguridad modernos.

Las contraseñas de aplicación son menos seguras que usar aplicaciones y servicios actualizados que utilicen estándares de seguridad modernos.

Antes de crear una contraseña de aplicación, debes comprobar si tu aplicación la necesita para iniciar sesión.

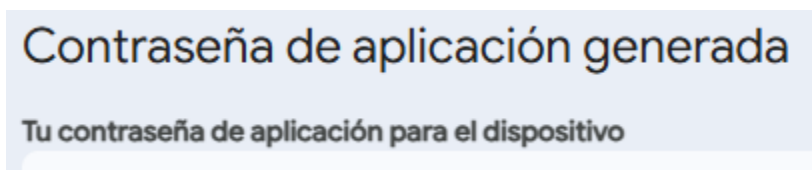
[Más información](#)

No tienes ninguna contraseña de aplicación.

Para crear una contraseña específica de la aplicación, escribe el nombre de la aplicación a continuación...

Nombre de la aplicación
prueba

Crear



Contraseña de aplicación generada

Tu contraseña de aplicación para el dispositivo

Y la contraseña que se muestra, es la que tenemos que usar en Kibana.

Email connector

Send email from your server.

Compatibility: Alerting Rules

Connector name

Prueba-Correo

Connector settings

Sender

Configure email accounts [Service](#)

Gmail

Host

smtp.gmail.com

Port

465

Secure

Authentication

☒ Require authentication for this server

Username

Password

Remember your Username and Password values. You must reenter them each time you edit the connector.

Save & test

Save

Username en authentication, es el correo, por ejemplo si es testkibana@gmail.com, en username se pondría testkibana sin el dominio. y en password, el código de la aplicación en google.

 **Edit connector**

Configuration Rules **Test**

1 Create an action

To Cc Bcc

Subject

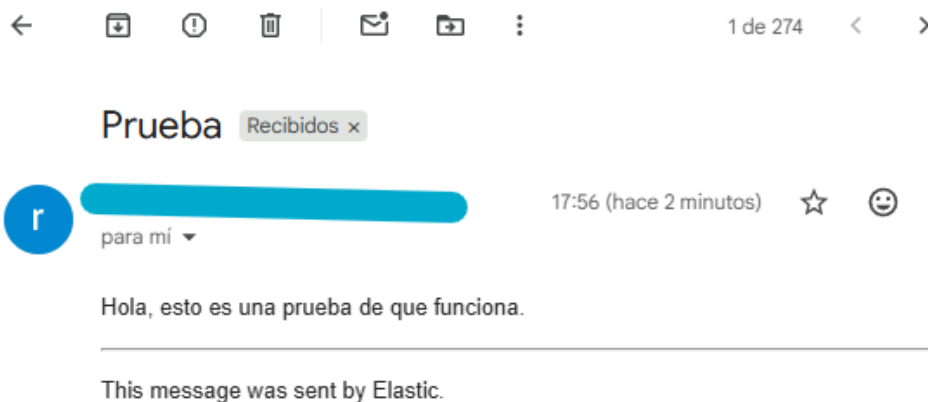
Message

2 Run the test

3 Results

✓ **Test was successful**
Ensure the results are what you expect.

La comprobación de que funciona.



El correo llega correctamente a la bandeja de entrada.

Paso 3.2.1 Mandar correo de Alerta cuándo se activa la regla.

En este vamos a ver cómo podemos hacer que Kibana nos mande el correo en caso de que se active o detecte una incidencia en el cliente windows.

No vamos a Rules.

The screenshot shows the Kibana Rules management page. At the top, there's a navigation bar with 'Stack Management' and 'Rules' tabs. Below this, the 'Rules' section is active, showing a list of rules. The interface includes filters for 'Rule state', 'Type', 'Action type', 'Last response', and 'Tags'. A 'Refresh' button is present. The status bar indicates 'Succeeded: 1', 'Failed: 0', and 'Warning: 0'. The rule list shows one rule named 'Brute Force Login Detection' with a status of 'Succeeded' and 'Enabled'. The table columns include Name, Last run, Notify, I..., D..., P50, S..., Last response, and S. The rule is currently enabled and has a success status.

Name	Last run	Notify	I...	D...	P50	S...	Last response	S.
Brute Force Login Detection	Dec 20, 2025 17:32:56p m 3 minutes ago	1	m	00:00	00:00	93%	Succeeded	Enabled

Damos en los tres puntos de la regla.

Enabled ▾

Rules

Documentation Settings Create rule

Detect conditions using rules.

Rules Logs

Search Rule state 0 Type 0 Action type 0 Last response 0 Tags 0 Refresh

● Succeeded: 1 ● Failed: 0 ● Warning: 0 Updated 3 minutes ago 5 m

1 rule Columns 10

☐	Name ↑	Last run ↕	Notify	I...	D...	P50 ↕	S...	Last response ↕	S. ↕
☐	Brute Force Login Detection Elasticsearch query	Dec 20, 2025 17:32:56p m 3 minutes ago		1	m	00:00	00:00	93%	● Succeeded Enabled ▾

Rows per page: 10 ▾

- Snooze
- Disable
- Clone rule
- Edit rule
- Update API key
- Run rule
- Delete rule

Editamos la regla, damos en edit rule.

Edit rule

container.id × host.hostname × host.id ×

host.name × kubernetes.pod.uid ×

▶ Test query

📄 Copy query

Check e... ? 1 minute

> Advanced options

Actions

Select a connector type

Cases

D3 Security

Email

IBM Resilient

Index

Jira

Empezamos a configurar la forma en la que llegará el correo.

[Practica ELK](#) © 2025 by [Rodríguez Rubén](#) is licensed under **CC BY-NC-ND 4.0**

Edit rule



Actions

▼

✉ Prueba-Correo

⊖

Email connector

Add connector

Prueba-Correo ▼

Action frequency

For each alert ▼ On status changes ▼

Run when Query matched ▼

☐ ✕

 If alert matches a query

☐ ✕

 If alert is generated during timeframe

To

Cc Bcc

Subject

✚

Message

✚

Elasticsearch query rule {{rule.name}} is active:

The screenshot shows the configuration for an email connector in Microsoft Sentinel. The 'Email connector' section has a dropdown set to 'Prueba-Correo'. The 'Action frequency' section has 'For each alert' selected. The 'Run when' section has 'Query matched' selected. There are two toggle switches, both turned off, for 'If alert matches a query' and 'If alert is generated during timeframe'. The 'To' field contains a redacted email address. The 'Subject' field is 'Posible ataque de fuerza bruta Windows'. The 'Message' field contains a template with placeholders for rule name, source IP, and failed attempt count.

Email connector [Add connector](#)

Prueba-Correo

Action frequency

For each alert On status changes

Run when Query matched

☐ If alert matches a query

☐ If alert is generated during timeframe

To Cc Bcc

[Redacted Email Address]

Subject

Posible ataque de fuerza bruta Windows

Message

Se ha detectado un posible ataque de fuerza bruta contra un equipo Windows.

Regla activada: {{rule.name}}

IP de origen sospechosa: {{context.source.ip}}

Número de intentos fallidos: {{context.failed.attempts}}

Se ha detectado un posible ataque de fuerza bruta contra un equipo Windows.

Regla activada: {{rule.name}}

Detalles del evento más reciente:

- IP de origen sospechosa: {{context.hits.0._source.source.ip}}
- Usuario: {{context.hits.0._source.user.name}}
- Equipo afectado: {{context.hits.0._source.host.hostname}}

Evento:

- Event ID: {{context.hits.0._source.event.code}}
- Tipo de logon: {{context.hits.0._source.winlog.logon.type}}
- Fecha: {{context.hits.0._source.@timestamp}}

Revisar inmediatamente en Kibana.

El texto que funciona mejor para los correos.

Vamos a probar la regla.

```
(ruben@rrg)-[~]
$ for i in {1..3}; do smbclient -L //192.168.65.143 -U fakeuser <<< "badpass"; done
Password for [WORKGROUP\fakeuser]:
session setup failed: NT_STATUS_LOGON_FAILURE
Password for [WORKGROUP\fakeuser]:
session setup failed: NT_STATUS_LOGON_FAILURE
Password for [WORKGROUP\fakeuser]:
session setup failed: NT_STATUS_LOGON_FAILURE
```

Desde kali linux vamos a hacer ruido, este comando hace peticiones fake al servicio smb de windows cliente, por lo tanto debe activarse la regla.

Si vamos a la app desde la regla creada, nos irá al discover con la consulta.

Brute_FORCE

Type **Elasticsearch query** API key owner **elastic**

[View in app](#) [Refresh](#) [Edit](#) [...](#)

Rule is **Enabled** [v](#)

27 executions in the last 24 hr

Last response 10 seconds ago

- Succeeded

[Notify when alerts generated](#)

Definition

Rule type Elasticsearch query

Actions [Prueba-Correo](#) [On status changes](#)

Description Alert when matches are found during the latest query run.

Runs every 1 min

Conditions 0 conditions

Alerts **History**

Columns 10 Sort fields 1 5 alerts Fields Updated now

Actions	Alert Status	Last updated	Started	Rule category
...	Active	Dec 20, 2025 @ 18:24:32.040	Dec 20, 2025 @ 18:24:32.040	Elasticsearch query

Discover [New](#) [Open](#) [Share](#) [Alerts](#) [Inspect](#) [Save](#)

winlogbeat-* [+](#) [-](#) [x](#) event.code: "4625" AND [x](#) Dec 20, 2025 @ 18:18:55.880 → Dec 20, 2025 @ 18:23:55.880 [x](#)

[Search field name](#) [+](#) [-](#) [x](#) 0

Available fields 353

- @timestamp
- agent.ephemeral_id
- agent.hostname
- agent.id
- agent.name
- agent.type
- agent.version
- data_stream.dataset
- data_stream.namespace
- data_stream.type
- ecs.version
- event.action
- event.category
- event.code
- event.created
- event.ingested

Visualizations [Auto interval](#) [No breakdown](#) [v](#)

Dec 20, 2025 @ 18:18:55.880 - Dec 20, 2025 @ 18:23:55.880 (interval: Auto - 5 seconds)

Documents (6) **Patterns** **Field statistics** [Sort fields](#) 1 [v](#) [x](#)

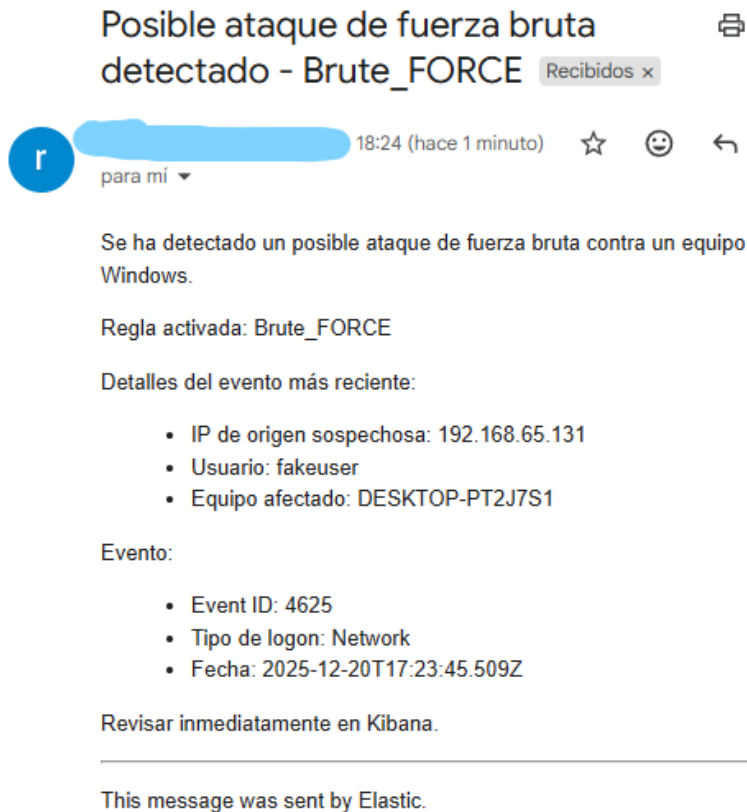
[Get the best look at your search results](#) [x](#)

Add relevant fields, reorder and sort columns, resize rows, and more in the document table.

[Take the tour](#) [Dismiss](#)

@timestamp	Document
Dec 20, 2025 @ 18:23:45.509	event.code 4625 winlog.logon.type Network @timestamp Dec 20, 2025 @ 18:23:45.509 agent.ephemeral_id 10c6fcfb-692f-4d70-add4-32e366ad638 5 agent.hostname DESKTOP-PT2J7S1 agent.id 83b62320-6c55-4f37-b3bf-62b3fcb90...
Dec 20, 2025 @ 18:23:45.481	event.code 4625 winlog.logon.type Network @timestamp Dec 20, 2025 @ 18:23:45.481 agent.ephemeral_id 10c6fcfb-692f-4d70-add4-32e366ad638 5 agent.hostname DESKTOP-PT2J7S1 agent.id 83b62320-6c55-4f37-b3bf-62b3fcb90...

En la captura anterior, vemos el registro Active, significa que se ha detectado una incidencia, por lo cuál, debemos de recibir un correo.



Y como se aprecia en la imagen, tenemos el correo, y la ip sospechosa, nos la muestra.

Si vamos a Kali....

```
(ruben@rrg)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:da:81:fd brd ff:ff:ff:ff:ff:ff
    inet 192.168.65.131/24 brd 192.168.65.255 scope global dynamic noprefixroute eth0
        valid_lft 1320sec preferred_lft 1320sec
    inet6 fe80::7abb:483f:af1e:94d8/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:c0:24:eb:4a brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
```

Efectivamente, tenemos el servicio bien configurado con la pertinente regla.

Aunque todo no es bueno, debemos de optimizar la sensibilidad de detección de ip posiblemente sospechosas.

En el siguiente paso, afinamos este apartado.

Paso 3.2.1.1 Configuración de los parámetros para evitar falsos positivos.

Cómo bien se describió en el último apartado, debemos de afinar la regla.

Para ello, regresamos a rules.

The screenshot shows the configuration page for a rule named "Brute_FORCE". At the top, there are buttons for "View in app", "Refresh", "Edit", and a menu icon. Below the title, it indicates the rule type is "Elasticsearch query" and the API key owner is "elastic".

The left sidebar shows the rule's status: "Rule is Enabled" with a dropdown arrow, "105 executions in the last 24 hr", "Last response Succeeded" (1 seconds ago), and a notification bell icon with the text "Notify when alerts generated".

The main content area is titled "Definition" and contains the following details:

- Rule type:** Elasticsearch query
- Actions:** Prueba-Correo (On status changes)
- Description:** Alert when matches are found during the latest query run.
- Runs every:** 1 min
- Conditions:** 0 conditions

At the bottom, there are tabs for "Alerts" and "History".

Debemos de cambiar la condición.

Set the group, threshold, and time window ⓘ

WHEN count()

GROUPED top 1 'host.hostname'
OVER

IS ABOVE 10

FOR THE 5 minutes
LAST

Set the number of documents to send ⓘ

SIZE 100

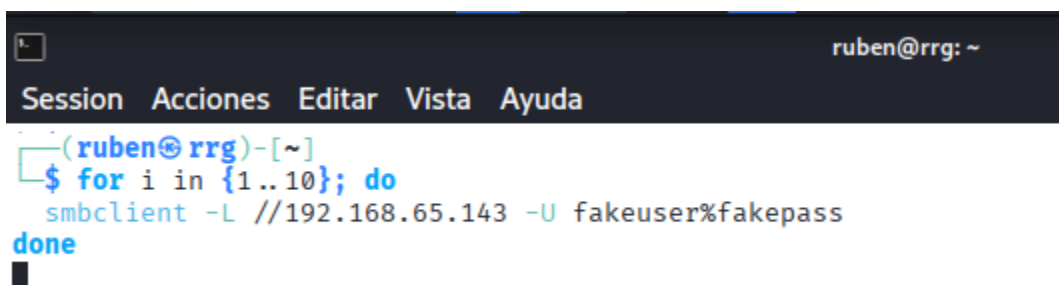
☒ Exclude matches from previous runs

Add more fields to alert details

Select fields

Esta decisión se toma con el objetivo de **evitar solapamientos y falsos positivos**, ya que múltiples intentos fallidos provenientes de distintas direcciones IP o usuarios podrían coincidir temporalmente, pero no representar un ataque real contra un mismo equipo.

Guardamos, y volvemos a ejecutar el comando en Kali.



```
ruben@rrg: ~  
Session Acciones Editar Vista Ayuda  
(ruben@rrg)-[~]  
$ for i in {1..10}; do  
smbclient -L //192.168.65.143 -U fakeuser%fakepass  
done
```

(Esto crea intentos falsos a smb).

```
smbclient -L //192.168.65.143 -U fakeuser%fakepass
done
```

```
session setup failed: NT_STATUS_LOGON_FAILURE
session setup failed: NT_STATUS_LOGON_FAILURE
session setup failed: NT_STATUS_LOGON_FAILURE
session setup failed: NT_STATUS_LOGON_FAILURE
session setup failed: NT_STATUS_LOGON_FAILURE
session setup failed: NT_STATUS_LOGON_FAILURE
session setup failed: NT_STATUS_LOGON_FAILURE
session setup failed: NT_STATUS_LOGON_FAILURE
session setup failed: NT_STATUS_LOGON_FAILURE
session setup failed: NT_STATUS_LOGON_FAILURE
```

```
(ruben@rrg)-[~]
$ for i in {1..10}
```

```
do
smbclient -L //192.168.65.143 -U fakeuser%fakepass
done
```

```
session setup failed: NT_STATUS_LOGON_FAILURE
session setup failed: NT_STATUS_LOGON_FAILURE
session setup failed: NT_STATUS_LOGON_FAILURE
session setup failed: NT_STATUS_LOGON_FAILURE
session setup failed: NT_STATUS_LOGON_FAILURE
session setup failed: NT_STATUS_LOGON_FAILURE
session setup failed: NT_STATUS_LOGON_FAILURE
session setup failed: NT_STATUS_LOGON_FAILURE
session setup failed: NT_STATUS_LOGON_FAILURE
session setup failed: NT_STATUS_LOGON_FAILURE
```

```
(ruben@rrg)-[~]
$ for i in {1..10}
```

```
do
smbclient -L //192.168.65.143 -U fakeuser%fakepass
done
```

```
session setup failed: NT_STATUS_LOGON_FAILURE
session setup failed: NT_STATUS_LOGON_FAILURE
session setup failed: NT_STATUS_LOGON_FAILURE
session setup failed: NT_STATUS_LOGON_FAILURE
session setup failed: NT_STATUS_LOGON_FAILURE
session setup failed: NT_STATUS_LOGON_FAILURE
session setup failed: NT_STATUS_LOGON_FAILURE
session setup failed: NT_STATUS_LOGON_FAILURE
session setup failed: NT_STATUS_LOGON_FAILURE
session setup failed: NT_STATUS_LOGON_FAILURE
```

```
for i in {1..10}
```

```
do
```

```
smbclient -L //192.168.65.143 -U fakeuser%fakepass
```

```
done
```


SOLO EN ÁMBITO CONTROLADO, nunca contra terceros

Laboratorio propio
Máquinas virtuales
Red aislada
Finalidad educativa

Ahora, vamos a Kibana.

The screenshot shows the Elastic Kibana interface. On the left is a sidebar with navigation links: Management, Ingest, Data, Alerts and Insights, Reporting, Machine Learning, Watcher, Maintenance Windows, Security, and Users. The main panel displays the configuration for a rule named 'Brute_FORCE'. The rule is an 'Elasticsearch query' type, owned by 'elastic', and is currently 'Enabled'. It has 109 executions in the last 24 hours. The last response was 'Succeeded' a minute ago. The definition shows a rule type of 'Elasticsearch query', a description 'Alert when matches are found during the latest query run.', a frequency of '1 min', and '0 conditions'. The actions are configured to send an email to 'Prueba-Correo' on status changes. Below the configuration, the 'Alerts' tab is active, showing a table of 7 alerts. The table has columns for Actions, Alert Status, Last updated, Started, and Rule category. The first two alerts are 'Active', and the remaining five are 'Recovered'.

Actions	Alert Status	Last updated	Started	Rule category
...	Active	Dec 20, 2025 @ 19:50:13.747	Dec 20, 2025 @ 19:44:58.749	Elasticsearch query
...	Active	Dec 20, 2025 @ 19:44:40.728	Dec 20, 2025 @ 19:39:04.740	Elasticsearch query
...	Recovered	Dec 20, 2025 @ 18:49:43.954	Dec 20, 2025 @ 18:24:32.040	Elasticsearch query
...	Recovered	Dec 20, 2025 @ 18:23:29.034	Dec 20, 2025 @ 18:19:17.032	Elasticsearch query
...	Recovered	Dec 20, 2025 @ 18:18:14.027	Dec 20, 2025 @ 18:11:56.033	Elasticsearch query
...	Recovered	Dec 20, 2025 @ 18:10:53.039	Dec 20, 2025 @ 18:07:44.053	Elasticsearch query

Se activó, por lo que se detectó, ya que al tener el umbral en 10 como tope, pues ha saltado, hicimos muchos intentos.

☐ yo 19:52
Posible ataque de fuerza bruta detectado - Brute_FORCE
Se ha detectado un posible ataque de fuerza bruta contra u... ☆

Alerts History

Columns 10 Sort fields 1 8 alerts Fields Updated 20 seconds ago

☐	Actions	Alert Status	Last updated	Started	Rule category
☐	...	Active	Dec 20, 2025 @ 19:52:04.726	Dec 20, 2025 @ 19:52:04.726	Elasticsearch query
☐	...	Active	Dec 20, 2025 @ 19:51:16.724	Dec 20, 2025 @ 19:44:58.749	Elasticsearch query
☐	...	Active	Dec 20, 2025 @ 19:44:40.728	Dec 20, 2025 @ 19:39:04.740	Elasticsearch query

19:52.

Funciona, esto nos demuestra que la regla funciona.

19:52 (hace 1 minuto) ☆ 😊 ↩

para mí ▼

Se ha detectado un posible ataque de fuerza bruta contra un equipo Windows.

Regla activada: Brute_FORCE

Detalles del evento más reciente:

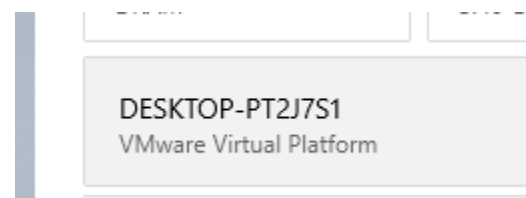
- IP de origen sospechosa: 192.168.65.131
- Usuario: fakeuser
- Equipo afectado: DESKTOP-PT2J7S1

Evento:

- Event ID: 4625
- Tipo de logon: Network
- Fecha: 2025-12-20T18:49:36.686Z

Revisar inmediatamente en Kibana.

This message was sent by Elastic.



(El id de la vm windows que teníamos enrolada, para evitar confusiones).

IMPORTANTE: Tras los 30 días, aunque esté montado en contenedores de Docker, y la vm apagada, cuándo iniciemos, se bloquea Kibana, y es porque la licencia, o sea, la prueba de 30 días, ha expirado.

Con el siguiente comando, todo en una línea, podremos volver a tener el ELK activo, con la licencia gratuita claro, no podremos hacer más lo de Correo por ejemplo.

```
rrodgar@rrgelk:~$ curl -k -X POST "https://192.168.65.20:9200/_license/start_basic?acknowledge=true" -u elastic [REDACTED]
```

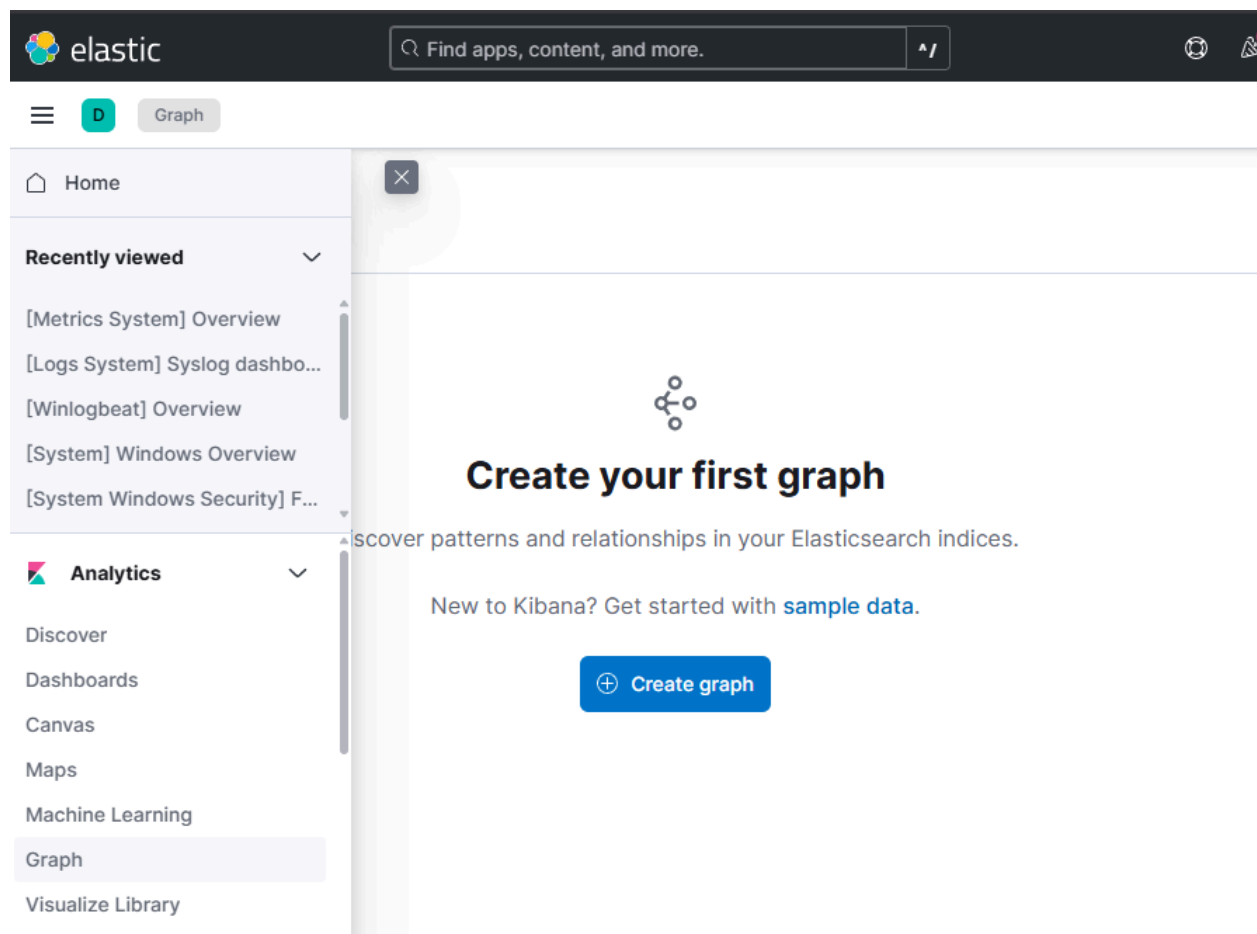
```
rrodgar@rrgelk:~$ curl -k -X POST "https://192.168.65.20:9200/_license/start_basic?acknowledge=true" \
-u elastic [REDACTED]
{"acknowledged":true,"basic_was_started":true}rrodgar@rrgelk:~$ ^C
```

Paso 4º. Creación de Grafos.

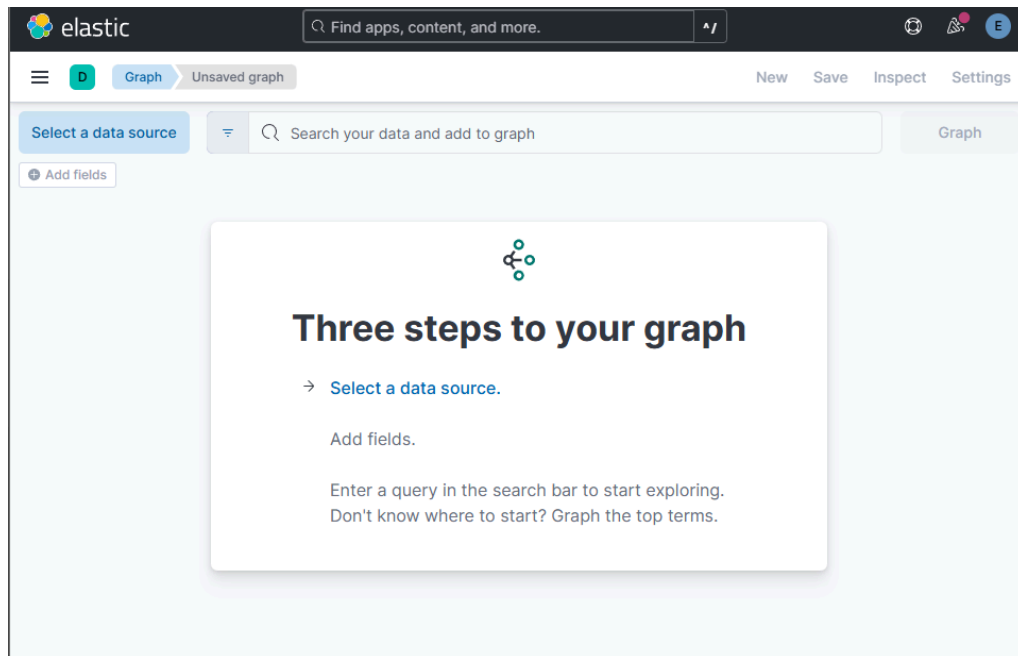
(Los gráficos sólo vienen incluido con la versión de paga, o en su defecto, con la prueba de 30 días, cómo bien se mencionó en el paso anterior.)

En Kibana podemos crear un **grafo interactivo** para visualizar relaciones entre usuarios, equipos y eventos de seguridad, lo cual es muy útil para un SOC para entender la infraestructura y detectar patrones sospechosos.

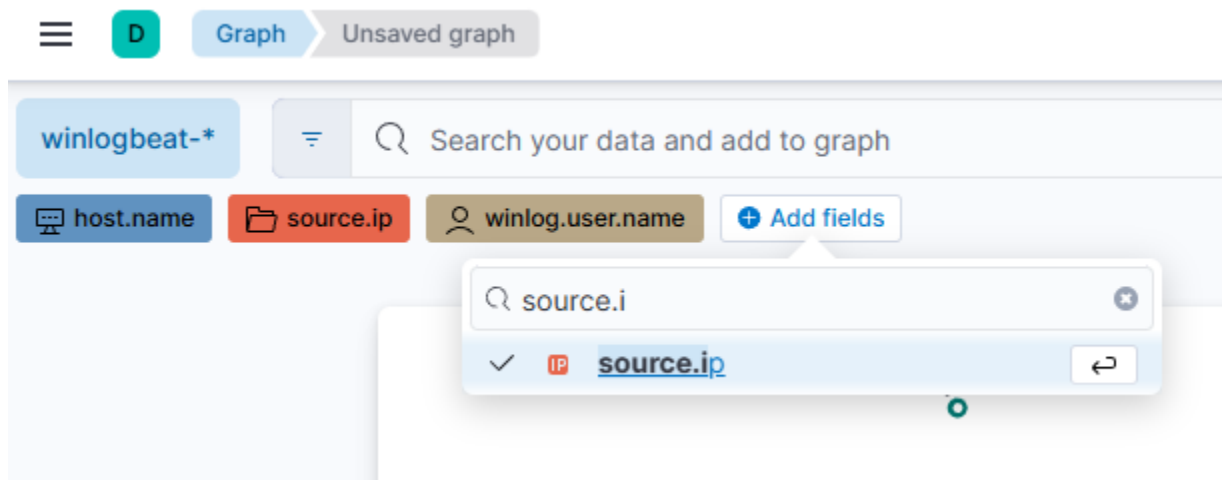
Nos vamos a Graph



Create graph

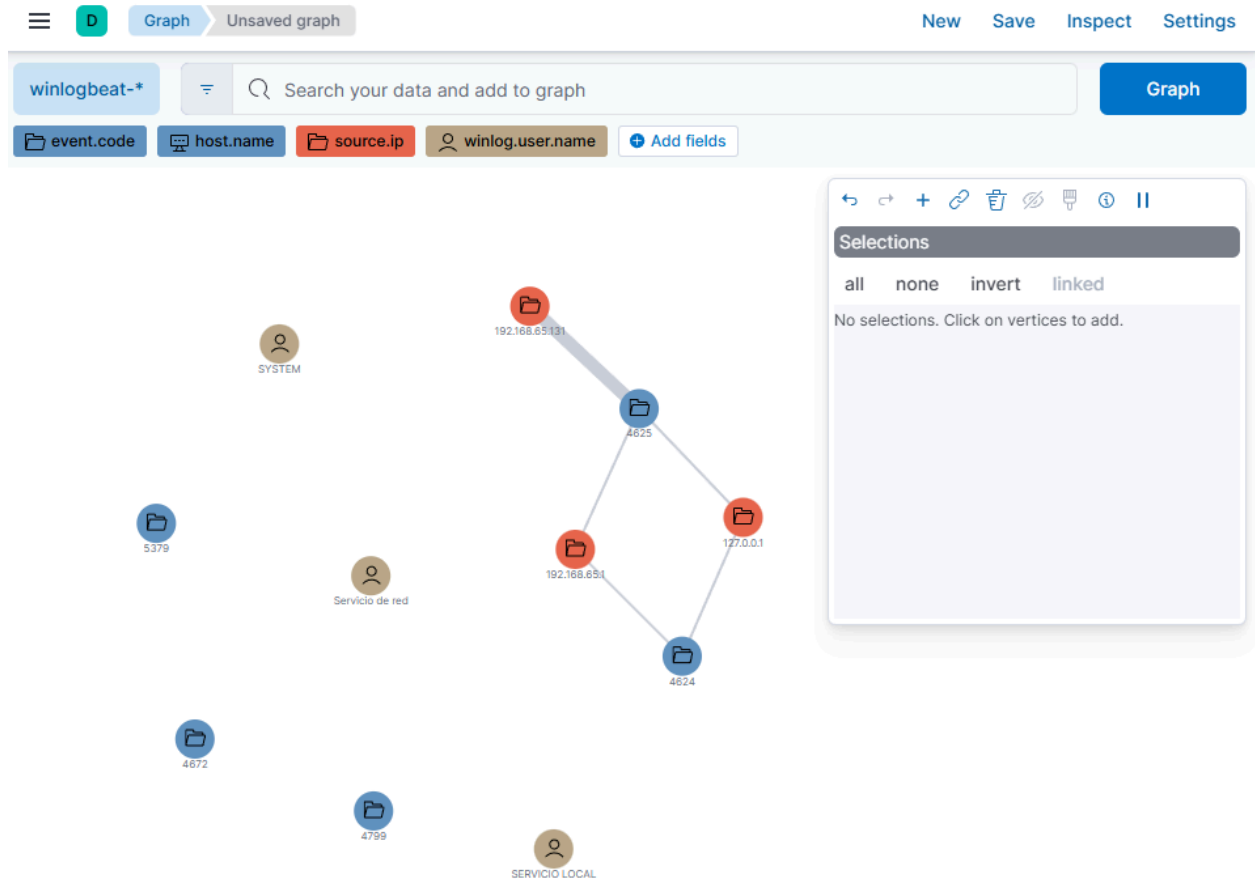


Tenemos que seleccionar los datos, para ello, seleccionaré los más relativos y lógicos para tener un análisis realista.



He seleccionado varios, entre los que tenemos el nombre del equipo, el usuario y la ip emisora.

Para crear el grafo, damos en Graph y..



Aquí tenemos un grafo, podemos ver la relación entre nodos por ejemplo.

Extensión: SIEM en DOCKER

Para facilitar la reproducción completa del laboratorio y de los incidentes de seguridad, se ha preparado un entorno basado completamente en contenedores Docker. Esta extensión permite:

Levantar todo el stack ELK (Elasticsearch, Logstash, Kibana) con un único comando.

- Crear contenedores víctimas (Ubuntu con SSH activo) y atacantes (Kali Linux) listos para generar incidentes.
- Configurar automáticamente Filebeat y Metricbeat para enviar logs al SIEM.
- Reproducir ataques de fuerza bruta o eventos repetitivos de manera controlada.
- Reiniciar y eliminar todo el entorno sin dejar residuos en el host.

Para ello, se ha usado un script que nos permite automatizar el proceso, ya que para ello, usaremos la base de lo que se vió anteriormente, aunque en este caso, se usará dos contenedores, y haremos que Kibana pinte cualquier evento en el dashboard.

Descripción del Entorno

Se ha creado un entorno completamente basado en Docker, compuesto por:

1. Stack ELK

- **Elasticsearch:** almacena e indexa los logs.
- **Logstash:** procesa los logs antes de enviarlos a Elasticsearch.
- **Kibana:** permite explorar, buscar y visualizar los eventos.

2. Contenedores adicionales

- **Víctima (Ubuntu):** con servicio SSH activo y generación de logs de seguridad.
- **Atacante (Kali Linux):** listo para ejecutar ataques reproducibles, como fuerza bruta SSH.

3. Filebeat

- Configurado para recolectar los logs de los contenedores y del host Docker.
- Envía automáticamente los logs a Logstash, que luego los envía a Elasticsearch.
- Permite que todos los eventos de seguridad sean visibles en Kibana.

4. Red interna

- Todos los contenedores están conectados a una **misma red Docker interna**, garantizando comunicación entre servicios y aislamiento del host, salvo los puertos expuestos necesarios (por ejemplo, Kibana 5601 y SSH de la víctima 2222).

5. Persistencia y orden

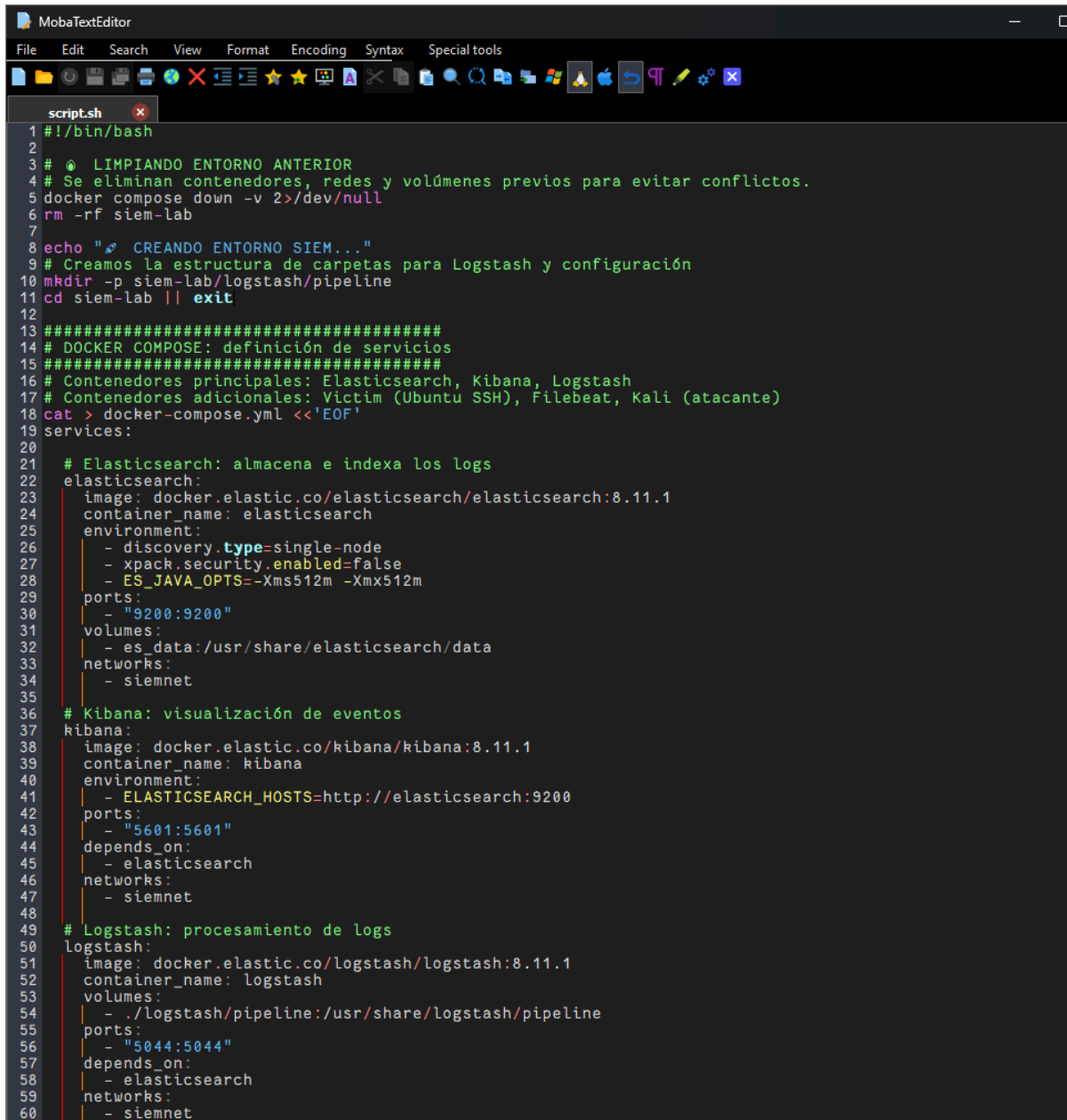
- Elasticsearch utiliza volúmenes persistentes para mantener datos aunque los contenedores se reinicien.
- Docker Compose define dependencias entre servicios para asegurar un arranque ordenado.

E.1 Preparación del fichero de implantación.

Para garantizar un despliegue ordenado, reproducible y fácil de mantener, se ha preparado un fichero de implantación basado en Docker Compose. Este fichero define todos los servicios necesarios del entorno —incluyendo Elasticsearch, Logstash, Kibana y los contenedores adicionales utilizados en el laboratorio— y permite levantar el sistema completo con un único comando.

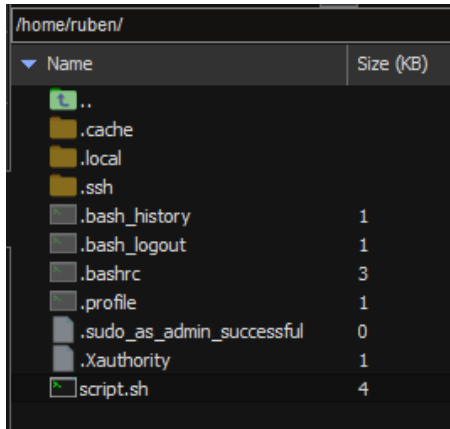
La preparación del fichero incluye la definición de redes, volúmenes persistentes, parámetros de configuración y dependencias entre servicios, asegurando que el SIEM se inicie de forma coherente y que los componentes puedan comunicarse correctamente desde el primer momento.

Empezamos por crear un pequeño script, que crea los ficheros necesarios junto a docker compose necesario para levantar el conjunto completamente.



```
1 #!/bin/bash
2
3 # LIMPIANDO ENTORNO ANTERIOR
4 # Se eliminan contenedores, redes y volúmenes previos para evitar conflictos.
5 docker compose down -v 2>/dev/null
6 rm -rf siem-lab
7
8 echo "CREANDO ENTORNO SIEM..."
9 # Creamos la estructura de carpetas para Logstash y configuración
10 mkdir -p siem-lab/logstash/pipeline
11 cd siem-lab || exit
12
13 #####
14 # DOCKER COMPOSE: definición de servicios
15 #####
16 # Contenedores principales: Elasticsearch, Kibana, Logstash
17 # Contenedores adicionales: Victim (Ubuntu SSH), Filebeat, Kali (atacante)
18 cat > docker-compose.yml <<'EOF'
19 services:
20
21 # Elasticsearch: almacena e indexa los logs
22 elasticsearch:
23   image: docker.elastic.co/elasticsearch/elasticsearch:8.11.1
24   container_name: elasticsearch
25   environment:
26     - discovery.type=single-node
27     - xpack.security.enabled=false
28     - ES_JAVA_OPTS=-Xms512m -Xmx512m
29   ports:
30     - "9200:9200"
31   volumes:
32     - es_data:/usr/share/elasticsearch/data
33   networks:
34     - siemnet
35
36 # Kibana: visualización de eventos
37 kibana:
38   image: docker.elastic.co/kibana/kibana:8.11.1
39   container_name: kibana
40   environment:
41     - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
42   ports:
43     - "5601:5601"
44   depends_on:
45     - elasticsearch
46   networks:
47     - siemnet
48
49 # Logstash: procesamiento de logs
50 logstash:
51   image: docker.elastic.co/logstash/logstash:8.11.1
52   container_name: logstash
53   volumes:
54     - ./logstash/pipeline:/usr/share/logstash/pipeline
55   ports:
56     - "5044:5044"
57   depends_on:
58     - elasticsearch
59   networks:
60     - siemnet
```

Tras ello, hay que dar permisos de ejecución, para poder ejecutarlo.



Lo ejecutamos.

```
ruben@banco:~$ sudo nano script.sh
[sudo] password for ruben:
ruben@banco:~$ sudo chmod +x script.sh
ruben@banco:~$ sudo ./script.sh
🔧 LIMPIANDO ENTORNO ANTERIOR...
🔧 CREANDO ENTORNO SIEM...
🔧 LEVANTANDO CONTENEDORES...
[+] up 7/7
✓ Network siem-lab_siemnet Created 0.0s
✓ Container kali Created 0.1s
✓ Container elasticsearch Created 0.1s
✓ Container victim Created 0.1s
✓ Container logstash Created 0.0s
✓ Container kibana Created 0.0s
✓ Container filebeat Created 0.0s

☑ ENTORNO LISTO

🔗 Kibana: http://192.168.65.144:5601

Para atacar:
docker exec -it kali bash
apt update && apt install -y hydra
for i in {1..20}; do hydra -l root -p 1234 ssh://victim -t 4; done

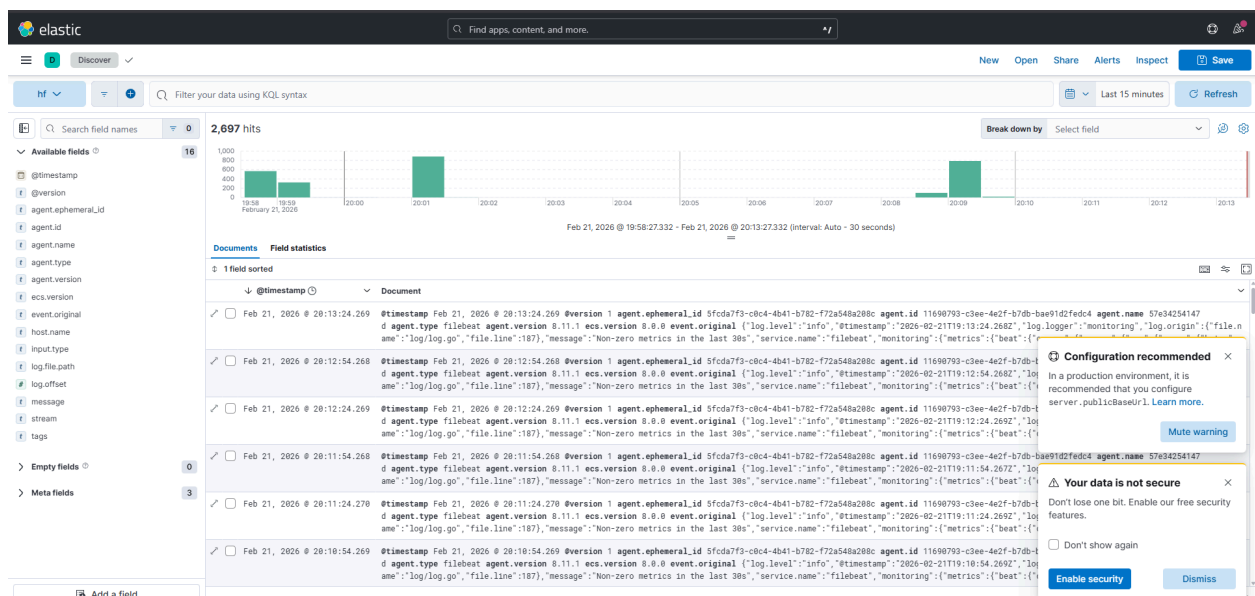
ruben@banco:~$
```

(Las imágenes se descargarán si no las tenemos).

Tras ello, ya tendremos el stack levantado correctamente.

```
ruben@banco:~$ docker ps -a
CONTAINER ID   IMAGE                                STATUS      PORTS                               COMMAND
57e34254147d   docker.elastic.co/beats/filebeat:8.11.1 Up 27 seconds          /usr/bin/
tini -- /u...
filebeat
8af45d8abd09   docker.elastic.co/kibana/kibana:8.11.1 Up 27 seconds          0.0.0.0:5601->5601/tcp, [::]:560
1->5601/tcp
kibana
abd4a50feca4   docker.elastic.co/logstash/logstash:8.11.1 Up 27 seconds          0.0.0.0:5044->5044/tcp, [::]:504
l/bin/dock...
4->5044/tcp, 9600/tcp
logstash
bbdc7ec4950c   ubuntu:22.04                       Up 28 seconds          0.0.0.0:2222->22/tcp, [::]:2222-
>22/tcp
victim
e2661548a046   docker.elastic.co/elasticsearch/elasticsearch:8.11.1 Up 28 seconds          0.0.0.0:9200->9200/tcp, [::]:920
0->9200/tcp, 9300/tcp
elasticsearch
dd70b3c5e368   kalilinux/kali-rolling              Up 28 seconds          bash
kali
ruben@banco:~$
```

Si nos vamos a la dirección del server y al puerto de Kibana, veremos que tenemos métricas.



E.2 Preparar la máquina atacante para el ataque.

En esta fase se prepara el contenedor atacante basado en Kali Linux con el objetivo de generar intentos de acceso fallidos contra la máquina víctima.

El atacante se encuentra en la misma red interna que el contenedor Ubuntu con SSH activo, lo que permite la comunicación directa. Tras acceder al contenedor, se instala la herramienta Hydra para simular ataques de fuerza bruta controlados.

Estos intentos generan eventos de autenticación fallida que serán capturados por Filebeat y enviados al SIEM para su análisis y visualización en Kibana.

Nos metemos en el contenedor de Kali, debemos de instalar una cosa.

```
ruben@banco:~$ docker exec -it kali bash
(root@dd70b3c5e368)-[/]
#
```

```
(root@ dd70b3c5e368)-[/]
# apt update && apt install -y hydra
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/contrib amd64 Packages [116 kB]
Get:3 http://kali.download/kali kali-rolling/non-free amd64 Packages [184 kB]
Get:4 http://kali.download/kali kali-rolling/main amd64 Packages [20.7 MB]
Get:5 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [1
1.8 kB]
Fetched 21.0 MB in 2s (12.0 MB/s)
All packages are up to date.
Installing:
hydra

Installing dependencies:
adduser          libhwy1t64      libtasn1-6
ca-certificates libicu76        libtdb1
dbus            libidn12        libtevent0t64
dbus-bin        libidn2-0       libthai-data
dbus-daemon     libigdgmm12     libthai0
dbus-session-bus-common libjansson4     libtheoradec2
dbus-system-bus-common libjbig0        libtheoraenc2
firebird4.0-common libjpeg62-turbo libtiff6
firebird4.0-common-doc libjson-c5      libtirpc-common
fontconfig      libjxl0.11      libtirpc3t64
fontconfig-config libk5crypto3    libtommath1
fonts-dejavu-core libkeyutils1    libtwolame0
fonts-dejavu-mono libkrb5-3       libunistring5
freetsd-common  libkrb5support0 liburiparser1
i965-va-driver  liblcms2-2     libutf8proc3
intel-media-va-driver libldap-common libva-drm2
krb5-locales    libldap2        libva-x11-2
libaom3         libldb2         libva2
libapparmor1    liblerc4        libvdpau1
libapr1t64      libllvm21       libvorbis0a
libaprutil1t64 liblmbd0        libvorbisenc2
libatomic1      libmariadb3     libvpl2
libavahi-client3 libmemcached11t64 libvpx12
libavahi-common-data libmongoc2=2    libwebclient0
```

Aquí instalamos hydra, el software que nos permitirá hacer fuerza bruta.

Tras ello, necesitamos crear un fichero pequeño con el cuál haremos fuerza bruta al contenedor de Ubuntu.

Instalaremos nano, ya que no viene en el contenedor de Kali.

```
(root@ dd70b3c5e368)-[/]
# apt install nano
Installing:
  nano

Installing dependencies:
  libgpm2 libncursesw6

Suggested packages:
  gpm hunspell

Summary:
  Upgrading: 0, Installing: 3, Removing: 0, Not Upgrading: 0
  Download size: 805 kB
  Space needed: 3397 kB / 7479 MB available

Continue? [Y/n] Y
0% [Waiting for headers]
```

Creamos el diccionario.

```
GNU nano 8.7 diccionario.txt *
1234
12345
admin
root
password
toor
qwerty
letmein
123456
root123
```


E.3 Ataque fuerza bruta desde Kali a Ubuntu.

En esta fase del laboratorio se simula un ataque de fuerza bruta desde el contenedor atacante (**Kali Linux**) hacia el contenedor víctima (**Ubuntu con SSH activo**) con el objetivo de generar eventos de autenticación fallida que serán registrados por el SIEM.

El propósito no es comprometer el sistema, sino **producir intentos de acceso repetidos** para que queden registrados en los logs y puedan visualizarse posteriormente en Kibana.

Para ello, usaremos el siguiente comando.

```
for i in {1..30}; do hydra -l -l root -p 1234 ssh://victim -t 4; done
```

Nota: Los comandos y técnicas mostrados en este laboratorio (como el uso de Hydra para fuerza bruta) **deben utilizarse exclusivamente en entornos controlados, propios o expresamente autorizados**.

La realización de ataques contra sistemas sin autorización constituye un delito según la legislación vigente y puede acarrear responsabilidades penales y civiles.

Este ejercicio tiene **finés exclusivamente educativos**, orientados al aprendizaje de detección y monitorización en entornos SIEM.

Aquí lo tenemos haciendo fuerza bruta al contenedor de Ubuntu.

```
(root@ dd70b3c5e368)-[/]
# for i in {1..30}; do hydra -I -l root -p 1234 ssh://victim -t
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do
litary or secret service organizations, or for illegal purposes (t
ding, these *** ignore laws and ethics anyway).

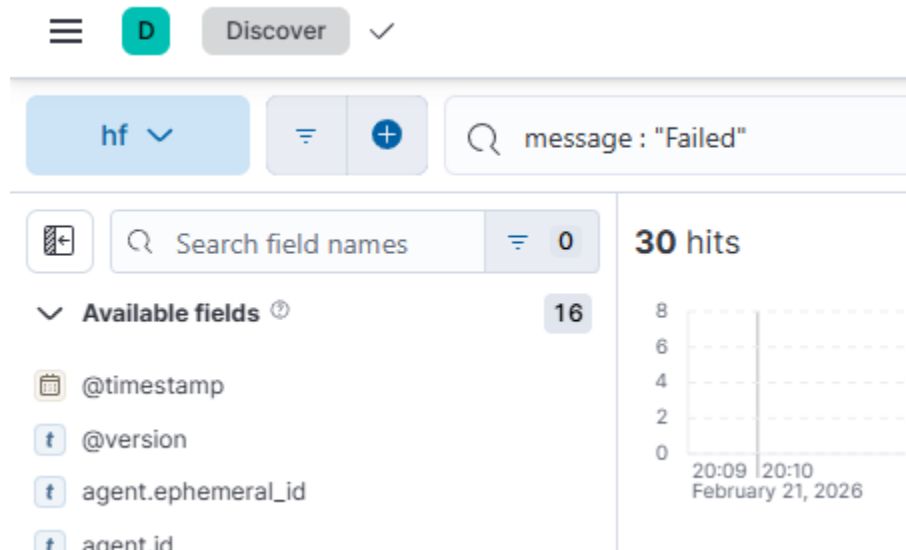
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 202
38
[DATA] max 1 task per 1 server, overall 1 task, 1 login try (l:1/p
r task
[DATA] attacking ssh://victim:22/
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 202
40
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do
litary or secret service organizations, or for illegal purposes (t
ding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 202
40
[DATA] max 1 task per 1 server, overall 1 task, 1 login try (l:1/p
r task
[DATA] attacking ssh://victim:22/
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 202
44
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do
litary or secret service organizations, or for illegal purposes (t
ding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 202
44
[DATA] max 1 task per 1 server, overall 1 task, 1 login try (l:1/p
r task
[DATA] attacking ssh://victim:22/
```

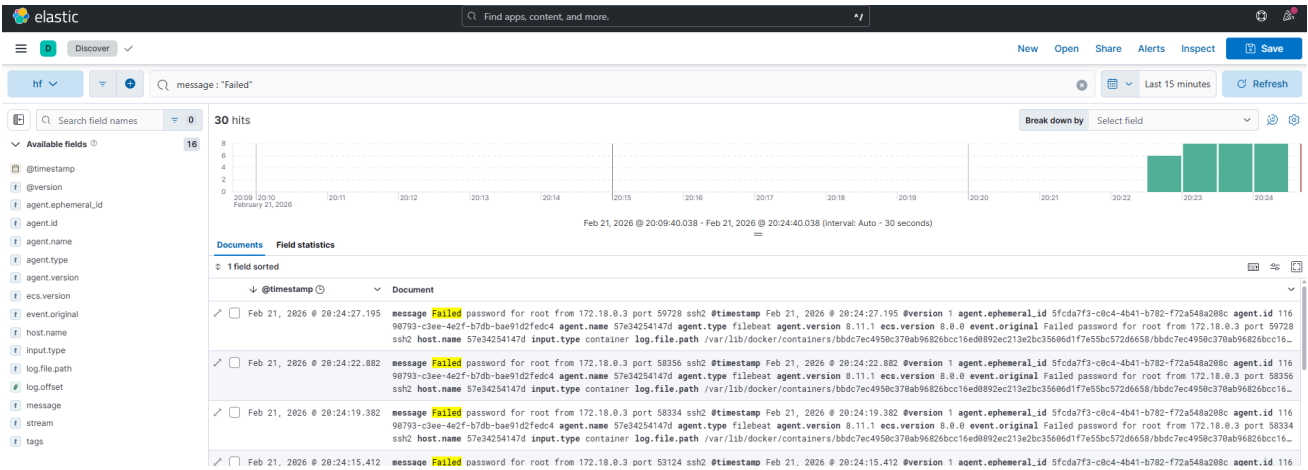
Por lógica, el pico de tráfico malicioso del contenedor Kali, debe de notarse en el discover de Kibana.

Si filtramos por la palabra clave de failed.



Aquí tenemos los registros, dónde el ELK ha detectado el pico de actividad maliciosa.





Expanded document

View: [Single document](#) [Surrounding documents](#) 1 of 4

Table JSON		
Search field names		
Actions	Field	Value
	k _id	9Pe2gZwBt07dis2YF0F7
	k _index	ssh-logs-2026.02.21
	# _score	-
	@timestamp	Feb 21, 2026 @ 20:38:37.761
	@version	1
	agent.ephemeral_id	5fcdaf7f3-c0c4-4b41-b782-f72a548a208c
	agent.id	11690793-c3ee-4e2f-b7db-bae91d2fedc4
	agent.name	57e34254147d
	agent.type	filebeat
	agent.version	8.11.1
	ecs.version	8.0.0
	event.original	Failed password for root from 172.18.0.3 port 41688 ssh2
	host.name	57e34254147d

Para confirmar que no hubo ninguna manipulación ni similar, vamos a ir al contenedor víctima de Ubuntu, y ver cuál es la ip interna.

iproute2 (para tener el comando ip a).

```
root@bbdc7ec4950c:/# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0@if123: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:fe:2e:76:b0:a9 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 172.18.0.4/16 brd 172.18.255.255 scope global eth0
        valid_lft forever preferred_lft forever
```

(172.18.0.4)

Ahora nos vamos al contenedor de Kali.

Instalamos el mismo paquete que ubuntu, para comprobar, que es la ip del atacante, tal y cómo se muestra en Kibana.

```
(root@ dd70b3c5e368)-[/]
# apt install iproute2
Installing:
iproute2

Installing dependencies:
libbpf1 libcap2-bin libmnl0 libpam-cap libxtables12

Suggested packages:
python3:any

Summary:
Upgrading: 0, Installing: 6, Removing: 0, Not Upgrading: 0
Download size: 1386 kB
Space needed: 4812 kB / 7471 MB available

Continue? [Y/n] Y
Get:1 http://http.kali.org/kali kali-rolling/main amd64 libbpf1 amd64 1:1.6.2-1+b1 [174 kB]
Get:2 http://http.kali.org/kali kali-rolling/main amd64 libmnl0 amd64 1.0.5-3+b1 [12.2 kB]
Get:3 http://kali.download/kali kali-rolling/main amd64 libxtables12 amd64 1.8.11-4 [32.1 kB]
Get:4 http://http.kali.org/kali kali-rolling/main amd64 libcap2-bin amd64 1:2.75-10+b5 [36.6 kB]
Get:5 http://kali.download/kali kali-rolling/main amd64 iproute2 amd64 6.18.0-1 [1114 kB]
Get:6 http://http.kali.org/kali kali-rolling/main amd64 libpam-cap amd64 1:2.75-10+b5 [16.5 kB]
Fetched 1386 kB in 1s (1483 kB/s)
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog based frontend cannot be used. at /usr/share/perl5/Debconf/FrontEnd/Dialog.pm line 79, <STDIN> line 6.)
debconf: falling back to frontend: Readline
```

```
(root@ dd70b3c5e368)-[/]
# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host proto kernel_lo
        valid_lft forever preferred_lft forever
2: eth0@if122: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 76:a1:01:08:0d:0d brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 172.18.0.3/16 brd 172.18.255.255 scope global eth0
        valid_lft forever preferred_lft forever

(root@ dd70b3c5e368)-[/]
#
```

(172.18.0.3)

Tal y cómo coincide, es la ip del que hizo el ataque de fuerza bruta.

Conclusión:

Con esta práctica se ha desplegado y configurado un stack ELK funcional sobre un entorno virtual, integrando un cliente Windows con Winlogbeat. Esto ha permitido centralizar los eventos del sistema, configurar alertas de seguridad (como detección de fuerza bruta) y recibir notificaciones automáticas por correo.

Se ha trabajado también la **seguridad en la transmisión de datos**, con HTTPS y gestión de credenciales de forma cifrada, y se han creado grafos para visualizar relaciones entre usuarios, equipos y eventos.